

F25-203-D-QSim

Project Team

Abdullah Mehmood	22i-0978
Abeer Noor	22i-1122
Aneeq Ahmed Malik	22i-1167

Session 2022-2026

Supervised by

Dr. Imran Ashraf



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

May, 2026

Contents

1	Introduction	1
1.1	Existing Solutions	1
1.1.1	Qiskit (IBM Quantum)	1
1.1.2	PennyLane (Xanadu)	2
1.1.3	Amazon Braket	2
1.1.4	Cirq (Google Quantum AI)	2
1.1.5	Gap Analysis	4
1.2	Problem Statement	4
1.3	Scope	5
1.3.1	OpenQASM 3.0 Scope	5
1.3.1.1	Comments and Version Control	5
1.3.1.2	Types and Casting	5
1.3.1.3	Gates	6
1.3.1.4	Built-in Quantum Instructions	6
1.3.1.5	Classical Instructions	6
1.3.1.6	Subroutines	7
1.3.1.7	Scoping of Variables	7
1.3.1.8	Directives	8
1.3.1.9	Circuit Timing	8
1.3.1.10	Pulse-level Descriptions	8
1.3.1.11	OpenPulse Grammar	8
1.3.1.12	Standard Library and Built-ins	9
1.3.1.13	Advanced Features	9
1.3.2	General Scope	10
1.4	Modules	10
1.4.1	IR Core	11
1.4.2	Backend Adapters	11
1.4.3	Job Manager	11
1.4.4	API Layer	11
1.5	Work Division	12
2	Project Requirements	13

2.1	Use-case	13
2.1.1	Use-case Diagram	13
2.1.2	Fully Dressed Use-cases	14
2.1.2.1	Use Case 1: Submit Quantum Circuit for Simulation . . .	14
2.1.2.2	Use Case 2: Parse Quantum Code	17
2.1.2.3	Use Case 3: Execute Quantum Circuit on Backend . . .	18
2.1.2.4	Use Case 4: Retrieve Simulation Results	21
2.1.2.5	Use Case 5: Manage Resource Allocation	23
2.1.2.6	Use Case 6: Schedule Jobs	25
2.1.2.7	Use Case 7: Monitor Job Status	27
2.1.2.8	Use Case 8: Handle Job Failure and Retry	31
2.1.2.9	Use Case 9: Cancel Running Job	33
2.1.2.10	Use Case 10: Monitor System Health and Performance .	35
2.1.2.11	Use Case 11: View Job History and Analytics	37
2.1.2.12	Use Case 12: Manage User Session	39
2.1.3	Event-Response Table	42
2.2	Functional Requirements	44
2.2.1	IR Core	44
2.2.2	Backend Adapters	44
2.2.3	Job Manager	44
2.2.4	API Layer	45
2.3	Non-Functional Requirements	46
2.3.1	Reliability	46
2.3.2	Usability	46
2.3.3	Performance	47
3	System Overview	49
3.1	Architectural Design	49
3.1.1	Environment Setup and Job Submission	49
3.1.2	Asynchronous Job Processing and Execution	51
3.1.3	Results and Metrics Persistence	53
3.1.4	Architectural Diagram	55
3.2	Design Models	57
3.2.1	Activity Diagram	57
3.2.2	Class Diagram	59
3.2.3	Class-level Sequence Diagram	59
3.3	Data Design	60
3.3.1	Overview	60
3.3.2	Data Transformation Pipeline	61
3.3.3	Major Data Entities	61

3.3.4	Data Storage Systems	62
3.3.5	Data Flow	63
3.3.6	Key Design Principles	63
3.4	Domain Model	63
4	Implementation and Testing	65
4.1	Algorithm Design	65
4.1.1	Main Simulation Pipeline	65
4.1.1.1	QASM Simulation Orchestration	65
4.1.1.2	QASM Parser	65
4.1.2	Component Factory Methods	67
4.1.2.1	Visitor and Backend Factories	67
4.1.3	Circuit Construction	67
4.1.3.1	AST Visitor Pattern	67
4.1.4	Simulation Execution	68
4.1.4.1	Backend Interface	68
4.1.5	Utility Functions	68
4.1.5.1	Qubit Count Inference	68
4.2	Documentation and SDK Resources	69
4.2.1	Software Documentation and Package Distribution	69
4.2.2	Resource Access Links	69
4.2.3	Installation Example	70
4.3	External APIs/SDKs	70
4.4	Testing Details	73
4.5	Unit Testing	73
4.5.1	Cirq Visitor Tests	73
4.5.2	Qiskit Visitor Tests	75
4.5.3	PennyLane Visitor Tests	77
5	Conclusions and Future Work	81
5.1	Conclusion	81
5.2	Future Work	82
5.2.1	Semantic-Aware Resource Scheduling (FAST-QSim)	82
5.2.2	Circuit Profiling and Resource Estimation	82
5.2.3	Job Cancellation and Preemption	82
5.2.4	Quota and Billing Reconciliation	83
5.2.5	Multi-Backend and GPU Simulation	83
5.2.6	Stateless Execution Latency	83
5.2.7	Research Paper: The FastQubit Platform	83
A	Code Examples	85

A.1	Qiskit Implementation	85
A.2	Cirq Implementation	85
A.3	PennyLane Implementation	86
A.4	OpenQASM 3.0 Code	86
A.5	QSim API Example	87
References		89

List of Figures

2.1	Use-case Diagram of QSim	14
3.1	Architectural Diagram of the System	55
3.2	Cloud Deployment Architecture	56
3.3	Activity Diagram-1	57
3.4	Activity Diagram-2	58
3.5	Class Diagram	59
3.6	Sequence Diagram-1	60
3.7	Sequence Diagram-2	60
3.8	Domain Model	64

List of Tables

1.1	Comparison of Existing Solutions	3
1.2	Comments and Version Control Constructs	5
1.3	Types and Casting Constructs	6
1.4	Gate Constructs	6
1.5	Built-in Quantum Instructions	6
1.6	Classical Instructions	7
1.7	Subroutine Constructs	7
1.8	Variable Scoping	7
1.9	Directive Constructs	8
1.10	Circuit Timing Constructs	8
1.11	Pulse-level Programming	8
1.12	OpenPulse Grammar	9
1.13	Standard Gates from OpenQASM 3 (<code>stdgates.inc</code>)	9
1.14	Advanced Features	9
1.15	Work Division Table	12
2.1	Event-Response Table	42
3.1	Key parameters of the <code>client.run()</code> API call	51
3.2	Supported simulation types and their characteristics	53
4.1	Documentation and SDK Resources for QSim	70
4.2	External APIs and SDKs Used in QSim Framework	73
4.3	Summary of Unit Test Coverage	73

CHAPTER 1

INTRODUCTION

This Final report outlines the development of a novel quantum simulator platform designed to enhance interoperability among existing quantum programming frameworks. The system aims to provide a unified environment where users can write quantum code in popular packages like Qiskit [4], Cirq [1], or PennyLane [3], convert it to a standard intermediate representation (OpenQASM 3.0) [2], and execute it across various simulators. Quantum computing is an emerging field with the potential to revolutionize computation by solving problems that are infeasible for classical computers. Quantum simulators provide a way for researchers and developers to test and design quantum algorithms on classical hardware before deploying them to actual quantum computers. Existing quantum simulators such as IBM’s Qiskit, Google’s Cirq, and PennyLane have laid the groundwork for this domain. However, these platforms operate independently and usually require users to adopt their specific coding frameworks. This document details the background, motivation, objectives, system architecture, workflow, and implementation details for building a robust, scalable, and secure quantum simulation platform.

1.1 Existing Solutions

The quantum computing landscape features several established frameworks developed by major technology companies and research institutions. This section examines four prominent solutions: IBM’s Qiskit, Xanadu’s PennyLane, Amazon’s Braket, and Google’s Cirq, critically assessing their capabilities and limitations to establish context for our unified quantum simulator platform.

1.1.1 Qiskit (IBM Quantum)

Qiskit is IBM’s open-source quantum framework offering a comprehensive software stack from high-level algorithm design to low-level pulse programming [4]. It features a modu-

lar architecture with specialized packages (Terra, Aer, Ignis, Aqua), advanced simulators (statevector, stabilizer, MPS), extensive gate libraries, noise modeling tools, and native IBM Quantum hardware integration.

However, Qiskit's proprietary circuit representation creates vendor lock-in, making migration to other frameworks difficult. The Python-based architecture introduces performance overhead for large circuits, while its IBM-centric design reduces framework neutrality. The complex API presents a steep learning curve, and limited interoperability with other quantum frameworks remains a persistent challenge.

1.1.2 PennyLane (Xanadu)

PennyLane is a quantum machine learning framework emphasizing hybrid quantum-classical computation [3]. It seamlessly integrates with ML libraries (TensorFlow, PyTorch, JAX) and provides automatic differentiation for quantum circuits. Its hardware-agnostic plugin architecture supports multiple backends, and the intuitive API follows ML conventions.

Despite these advantages, PennyLane's ML optimization restricts general-purpose quantum computing flexibility. The plugin architecture introduces compatibility challenges, and it offers limited low-level circuit control compared to Qiskit or Cirq. A smaller community results in fewer integrations, and documentation gaps exist for non-ML quantum applications.

1.1.3 Amazon Braket

Amazon Braket is AWS's managed quantum service providing unified access to multiple hardware providers (IonQ, Rigetti, OQC, D-Wave). It offers deep AWS integration (S3, CloudWatch, IAM), managed notebooks, hybrid algorithms, automatic resource provisioning, and pay-per-use pricing.

However, Braket creates strong AWS platform lock-in and has higher costs than self-hosted solutions. It requires continuous cloud connectivity, limiting offline development. Customization options are restricted, and AWS-specific API conventions may conflict with quantum computing best practices. Cost barriers make it less accessible for educational institutions.

1.1.4 Cirq (Google Quantum AI)

Cirq is Google's open-source framework designed for NISQ devices [1]. It provides fine-grained circuit control optimized for near-term quantum computing, native Google processor support, explicit qubit placement and timing control, hardware-aware noise modeling, and performance-optimized C++ backends.

Nevertheless, Cirq’s Google hardware-centric design reduces framework neutrality. The emphasis on low-level control creates a steep learning curve, and the API is less intuitive than higher-level frameworks. It provides a limited high-level algorithm library, has a smaller community than IBM’s ecosystem, and circuit representations are not easily portable to other frameworks.

Table 1.1: Comparison of Existing Solutions

System Name	System Overview	System Limitations
Qiskit (IBM Quantum)	Open-source framework with modular architecture, advanced simulators (statevector, stabilizer, MPS), extensive gate library, noise modeling, and IBM hardware integration.	Proprietary circuit representation creates vendor lock-in; Python performance overhead; IBM-centric optimization; limited interoperability; complex API; steep learning curve.
PennyLane (Xanadu)	Quantum ML framework with ML library integration (TensorFlow, PyTorch, JAX), automatic differentiation, hardware-agnostic plugins, and intuitive ML-style API.	ML-focused design restricts general-purpose flexibility; plugin compatibility challenges; limited low-level control; smaller community; performance variability; documentation gaps for non-ML use.
Amazon Braket	Managed AWS service with access to multiple hardware providers, AWS integration, managed notebooks, hybrid algorithms, and pay-per-use pricing.	Strong AWS lock-in; higher costs; requires cloud connectivity; restricted customization; AWS-specific conventions; geographic limitations; cost barriers for education.
Cirq (Google Quantum AI)	NISQ-focused framework with fine-grained control, Google processor support, hardware-aware noise modeling, and C++ simulator backends.	Google-centric design; steep learning curve; less intuitive API; limited algorithm library; smaller community; documentation assumes expertise; portability challenges.

1.1.5 Gap Analysis

Analysis of existing frameworks reveals critical gaps: **framework fragmentation** creates barriers to code sharing; **vendor lock-in** limits access to diverse quantum processors; **interoperability deficits** force code rewrites; **resource accessibility** challenges (especially cloud costs) limit educational access; **simulation diversity** is inadequate as no platform optimally supports all modes; and **educational barriers** from varied APIs hinder learning.

Our unified quantum simulator platform addresses these limitations by implementing OpenQASM 3.0 as a universal intermediate representation, enabling seamless execution across multiple frameworks and simulators. This eliminates vendor lock-in, reduces learning overhead, and democratizes access through distributed computing capabilities deployable on local infrastructure without cloud dependencies.

1.2 Problem Statement

Quantum computing holds immense promise, but the current landscape of quantum simulators presents significant barriers to widespread adoption and collaboration. Existing platforms like Qiskit, Cirq, and PennyLane each require users to learn and adhere to their proprietary coding frameworks, leading to vendor lock-in and fragmented development efforts. This silos knowledge and resources, making it difficult for researchers to share code or switch between tools without extensive rewrites. Additionally, the lack of interoperability hinders the ability to leverage the strengths of different simulators for various quantum circuit types. We are developing this system to address these limitations by creating a unified quantum simulator that supports code from multiple frameworks. The software solves the problem of framework dependency by introducing a converter to OpenQASM 3.0, allowing seamless execution across integrated simulators. This reduces the time and effort needed for code adaptation and promotes accessibility for a broader range of users. Furthermore, it tackles issues related to resource management in distributed environments, where simulations can be computationally intensive. By incorporating efficient scheduling, fault tolerance, and scalability features, the system ensures reliable performance for large-scale quantum simulations. The absence of such a versatile platform particularly affects regions like Pakistan, where access to proprietary quantum hardware and software is limited. Our solution aims to democratize quantum computing education and research by providing an open, cross-platform tool. This will enable local developers and academics to innovate without relying on external systems. Overall, the problem lies in the inefficiency and inaccessibility of current quantum simulation tools, which our software directly resolves through interoperability and robust backend support. The development of this system is crucial for fostering global collaboration in quantum technologies.

1.3 Scope

The scope of this project is divided into two main areas: the implementation of OpenQASM 3.0 language constructs and the general functionality of the quantum simulator platform. These are outlined below:

1.3.1 OpenQASM 3.0 Scope

- **Standard Intermediate Representation:** Leverages OpenQASM 3.0 as a standard intermediate representation (IR) for interoperability across quantum computing frameworks including Qiskit, Cirq, and PennyLane. [2]

1.3.1.1 Comments and Version Control

Language Construct	Phase
Single-line Comments (//) ↗	Iteration I
Multi-line Comments (/* */) ↗	Iteration I
Version String (OPENQASM 3.0) ↗	Iteration I
Include Statements ↗	Iteration I

Table 1.2: Comments and Version Control Constructs

1.3.1.2 Types and Casting

Language Construct	Phase
Identifiers ↗	Iteration I
Variables ↗	Iteration I
Quantum Types (qubit) ↗	Iteration I
Physical Qubits (\$0, \$1, ...) ↗	Excluded
Boolean Type (bool) ↗	Iteration I
Bit Type (bit) ↗	Iteration I
Integer Type (int) ↗	Iteration I
Unsigned Integer Type (uint) ↗	Iteration I
Float Type (float) ↗	Iteration I
Angle Type (angle) ↗	Iteration I
Complex Type (complex) ↗	Iteration II
Compile-time Constants (const) ↗	Iteration I
Literals (Integer, Float, Boolean) ↗	Iteration I
Arrays ↗	Iteration I
Duration Type (duration) ↗	Excluded

Language Construct	Phase
Stretch Type (stretch) ↗	Excluded
Aliasing ↗	Iteration I
Index Sets and Slicing ↗	Iteration I
Register Concatenation ↗	Iteration I
Classical Value Bit Slicing ↗	Excluded
Array Concatenation ↗	Iteration I
Type Casting ↗	Iteration II

Table 1.3: Types and Casting Constructs

1.3.1.3 Gates

Language Construct	Phase
Applying Gates ↗	Iteration I
Gate Broadcasting ↗	Iteration I
Parameterized Gates ↗	Iteration I
Hierarchical Gate Definitions ↗	Iteration I
Control Modifier (ctrl @) ↗	Iteration I
Negative Control Modifier (negctrl @) ↗	Iteration II
Multi-Control (ctrl(n) @) ↗	Iteration II
Inverse Modifier (inv @) ↗	Iteration I
Power Modifier (pow(k) @) ↗	Iteration II
Built-in U Gate ↗	Iteration I
Global Phase Gate (gphase) ↗	Iteration I

Table 1.4: Gate Constructs

1.3.1.4 Built-in Quantum Instructions

Language Construct	Phase
Reset Instruction ↗	Iteration I
Measurement Instruction ↗	Iteration I
Barrier Instruction ↗	Iteration I
Delay Instruction ↗	Excluded

Table 1.5: Built-in Quantum Instructions

1.3.1.5 Classical Instructions

Language Construct	Phase
Assignment Statements ↗	Iteration I
Arithmetic Operations (+, -, *, /) ↗	Iteration I
Comparison Operations (<, >, ==, !=) ↗	Iteration I
Logical Operations (&&, , !) ↗	Iteration I
Bitwise Operations (&, , ^, ~) ↗	Iteration II
Shift Operations («, ») ↗	Iteration II
If Statements ↗	Iteration I
If-Else Statements ↗	Iteration I
While Loops ↗	Iteration II
For Loops ↗	Iteration I
Break Statement ↗	Iteration II
Continue Statement ↗	Iteration II
Mathematical Functions ↗	Iteration II

Table 1.6: Classical Instructions

1.3.1.6 Subroutines

Language Construct	Phase
Subroutine Declaration (def) ↗	Iteration II
Return Statements ↗	Iteration II
Function Parameters ↗	Iteration II
Quantum Parameters ↗	Iteration II
Classical Parameters ↗	Iteration II
Reference Parameters ↗	Excluded

Table 1.7: Subroutine Constructs

1.3.1.7 Scoping of Variables

Language Construct	Phase
Global Scope ↗	Iteration I
Local Scope ↗	Iteration I
Gate Scope ↗	Iteration I
Subroutine Scope ↗	Iteration II
Block Scope ↗	Iteration II

Table 1.8: Variable Scoping

1.3.1.8 Directives

Language Construct	Phase
Pragma Directives ↗	Excluded
Annotations (@) ↗	Excluded
Input Declaration ↗	Iteration I
Output Declaration ↗	Iteration I

Table 1.9: Directive Constructs

1.3.1.9 Circuit Timing

Language Construct	Phase
Delay Instructions ↗	Excluded
Duration Literals ↗	Excluded
Stretch Type ↗	Excluded
Box Statement ↗	Excluded

Table 1.10: Circuit Timing Constructs

1.3.1.10 Pulse-level Descriptions

Language Construct	Phase
Calibration Blocks (defcal) ↗	Excluded
Cal Statement ↗	Excluded
Pulse Gates ↗	Excluded
Waveform Definition ↗	Excluded
Frame Definitions ↗	Excluded
Port Definitions ↗	Excluded

Table 1.11: Pulse-level Programming

1.3.1.11 OpenPulse Grammar

Language Construct	Phase
Pulse Instructions ↗	Excluded
Capture Instructions ↗	Excluded
Play Instructions ↗	Excluded
Set Phase Instructions ↗	Excluded

Language Construct	Phase
Shift Phase Instructions ↗	Excluded
Set Frequency Instructions ↗	Excluded
Shift Frequency Instructions ↗	Excluded

Table 1.12: OpenPulse Grammar

1.3.1.12 Standard Library and Built-ins

Gate Family	Iteration
Identity (id) ↗	Iteration I
Pauli Gates (x, y, z) ↗	Iteration I
Hadamard (h) ↗	Iteration I
Phase Gates (s, sdg, t, tdg) ↗	Iteration I
Rotation Gates (rx, ry, rz) ↗	Iteration I
Parameterized U Gate (u) ↗	Iteration I
Global Phase (gphase) ↗	Iteration I
CNOT (cx) ↗	Iteration I
Controlled Gates (cy, cz) ↗	Iteration I
SWAP Family (swap, cswap, iswap, sqrtiswap) ↗	Iteration I
Toffoli (ccx) ↗	Iteration II

Table 1.13: Standard Gates from OpenQASM 3
(stdgates.inc)

1.3.1.13 Advanced Features

Language Construct	Phase
Extern Functions ↗	Excluded
Memory Management ↗	Excluded
Low-level Classical Instructions ↗	Excluded
Hardware-specific Extensions ↗	Excluded
Quantum Error Correction ↗	Excluded

Table 1.14: Advanced Features

1.3.2 General Scope

- **Versatile Simulation Platform:** Develops a quantum simulator platform supporting format conversion to translate quantum code from multiple frameworks into OpenQASM 3.0 IR for unified execution.
- **Multiple Simulation Modes:** Offers various simulation backends including statevector, density matrix, Clifford+Hadamard (CH), matrix product state (MPS), and shot-based execution, selectable via a frontend interface.
- **Python Integration:** Provides comprehensive Python package for job submission, real-time monitoring, and result retrieval.
- **Distributed Computing:** Enables distributed computing via Kubernetes orchestration, offloading circuits to Q-Nodes with fair resource allocation across CPU, GPU, and high-memory pools.
- **Resource Management:** Implements advanced resource management with priority-based scheduling, queue management, and Kubernetes Job orchestration.
- **Performance Optimization:** Optimizes performance with predictive resource estimation based on IR metrics to enhance scheduling efficiency.
- **Fault Tolerance:** Ensures system reliability with Kubernetes probes, automatic rescheduling, retries, and checkpointing mechanisms.
- **Monitoring and Logging:** Utilizes Prometheus, and Grafana for comprehensive logging, monitoring, and alerting capabilities.
- **Resource Tracking:** Tracks resource usage (CPU, GPU, memory, storage) for efficiency optimization and billing with quota enforcement.
- **Backend Support:** Supports multiple simulation backends including Qiskit Aer and Cirq Simulator, aligning with diverse hardware demands.
- **Job Management:** Manages job submission with OpenQASM 3.0 IR encoding, authentication, and durable queue systems.

1.4 Modules

This section outlines the key modules of the QSim system, each responsible for specific functionalities. These modules ensure modular design for maintainability and scalability.

1.4.1 IR Core

This module parses OpenQASM 3.0 input into an Abstract Syntax Tree (AST), validates syntax/semantics, and translates to SDK-specific circuits, while handling unsupported constructs and errors gracefully.

1. Generate AST from OpenQASM 3.0 code.
2. Raise detailed errors for unsupported constructs or invalid syntax, including line references.
3. Translate parsed AST to Qiskit, Cirq, or PennyLane circuits.

1.4.2 Backend Adapters

This module provides framework-specific adapters to execute translated circuits on simulators and retrieve formatted results, supporting GPU acceleration where applicable.

1. Adapter for Qiskit framework integration and result formatting (e.g., statevectors, counts).
2. Adapter for Cirq framework integration and result formatting.
3. Adapter for PennyLane framework integration and result formatting.
4. Handle execution errors and serialize results for storage (e.g., in S3).

1.4.3 Job Manager

This module handles resource allocation, job queuing/orchestration, status monitoring, failure recovery, and cancellation across Kubernetes nodes.

1. Resource management and allocation (CPU, memory, GPU) with quota checks.
2. Node orchestration, queuing (RabbitMQ), and execution scheduling.
3. Real-time job status monitoring and updates (e.g., queued, running, completed).
4. Failure handling, including retries, logging, and notifications.
5. Job cancellation with graceful shutdown and resource cleanup.

1.4.4 API Layer

This module exposes a user-friendly Python package and REST/GraphQL endpoints for authentication, job submission, status queries, result retrieval, and history analytics. For a demonstration of the QSim API, see [Appendix A.5](#).

1. Pip-installable Python package with SDK-like interface.
2. Authentication and session management (JWT tokens, expiration).
3. Endpoints for job submission, validation, and queuing.
4. Status querying (single/batch, WebSocket real-time updates).
5. Result retrieval with deserialization and job history/analytics (filtering, pagination).

1.5 Work Division

For each module and respective feature, assign responsibility to a team member

Table 1.15: Work Division Table

Name	Registration	Responsibility/ Module / Feature
Abdullah Mehmood	22i-0978	(Module 1 - Feat 1), (Module 2 - Feat 2), (Module 3 - Feat 2)
Abeer Noor	22i-1122	(Module 1 - Feat 1), (Module 2 - Feat 1), (Module 3 - Feat 1, 3)
Aneeq Ahmed Malik	22i-1167	(Module 1 - Feat 1), (Module 2 - Feat 3), (Module 4 - Feat 1, 2)

CHAPTER 2

PROJECT REQUIREMENTS

This section outlines the necessary requirements for the successful completion of the project.

2.1 Use-case

This section introduces the use case models for QSim, encompassing high-level diagrams that depict user interactions and system responses, followed by fully dressed narratives that detail scenarios, actors, preconditions, and extensions to ensure comprehensive coverage of system behaviors.

2.1.1 Use-case Diagram

This subsection illustrates the use case diagram for QSim, offering a high-level visual representation of the primary actors—such as Users and System Administrators—and their interactions with core functionalities like job submission, circuit execution, result retrieval, and system monitoring, thereby encapsulating the system’s behavioral requirements. (see [Figure 2.1](#))

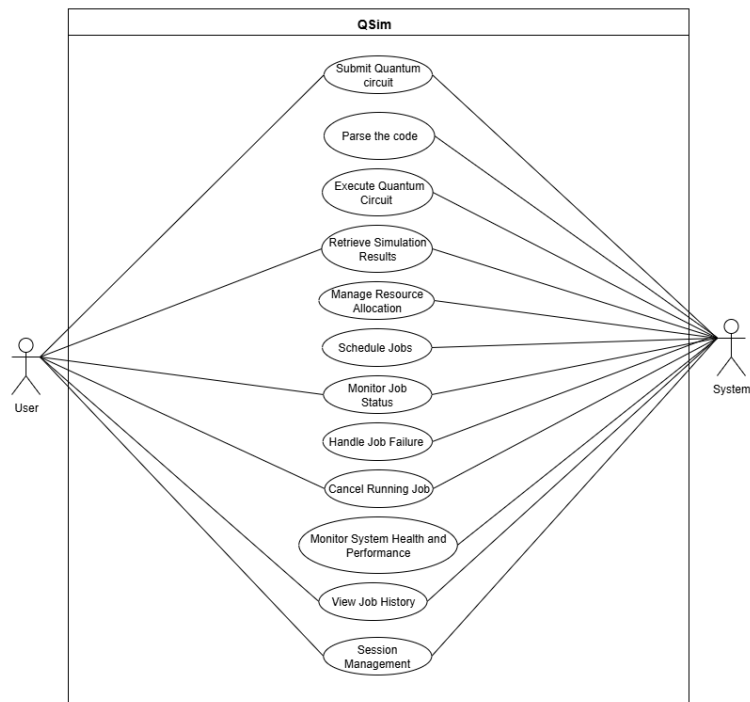


Figure 2.1: Use-case Diagram of QSim

2.1.2 Fully Dressed Use-cases

This subsection provides in-depth elaborations of the fully dressed use cases (UC-01 through UC-12), articulating the main success paths, extensions for error handling, special requirements, and technology integrations to guide precise implementation and testing.

2.1.2.1 Use Case 1: Submit Quantum Circuit for Simulation

Use Case ID: UC-01

Use Case Name: Submit Quantum Circuit

Actors

- **Primary:** User

Stakeholders and Interests

- **User:** Seeks to execute quantum circuits in preferred frameworks.
- **System Administrator:** Requires reliable job submission and tracking.

Preconditions

- User possesses valid authentication credentials.
- User has a valid OpenQASM 3.0 IR.
- System has available compute resources.
- Backend simulator is operational.

Postconditions

- **Success Guarantee**
 - Job is submitted and queued successfully.
 - User receives a unique job ID.
 - Job status is set to “queued”.
 - IR is translated into a quantum circuit in Qiskit, Cirq, or PennyLane.
- **Minimal Guarantee**
 - System logs the submission attempt.
 - User receives an error message if submission fails.

Main Success Scenario

1. User submits the circuit and configuration parameters via the system API.
2. System authenticates the user and validates inputs.
3. System converts the IR (OpenQASM 3.0) to a quantum circuit in Qiskit, Cirq, or PennyLane.
4. Job manager records the job, assigns a unique ID, and attaches metadata.
5. Job is queued for execution.
6. System returns the job ID to the user.

Extensions

- **3a. Authentication Fails**
 - 3a1. System rejects invalid credentials.
 - 3a2. System returns an authentication error.

- **4a. Invalid Circuit Syntax**
 - 4a1. System detects syntax errors.
 - 4a2. System returns a validation error with details.
- **5a. Translation to SDK failed**
 - 5a1. IR core encounters an unsupported construct.
 - 5a2. System logs the translation error.
 - 5a3. System returns an error indicating the unsupported feature.
- **6a. OpenQASM 3.0 Validation Fails**
 - 6a1. System detects invalid IR.
 - 6a2. System returns a validation error.
- **7a. Queue is Full**
 - 7a1. System checks queue capacity.
 - 7a2. System returns a “queue full” error.
 - 7a3. User is advised to retry later.
- **8a. Insufficient User Quota**
 - 8a1. System checks user resource quota.
 - 8a2. System rejects the submission.
 - 8a3. System returns a “quota exceeded” error.

Special Requirements

- Enable concurrent job submissions.
- Ensure secure API communication via HTTPS.

Technology and Data Variations

- **Output Formats:** Qiskit QuantumCircuit, Cirq Circuit, PennyLane QNode.
- **IR Format:** OpenQASM 3.0.
- **Queue System:** RabbitMQ.
- **Database:** PostgreSQL.

2.1.2.2 Use Case 2: Parse Quantum Code

Use Case ID: UC-02

Use Case Name: Parse Quantum Circuit Code

Actors

- **Primary:** System
- **Secondary:** User

Stakeholders and Interests

- **User:** Ensures code validity pre-execution.
- **System Administrator:** Requires error handling and logging.

Preconditions

- User authenticated.
- Valid IR code provided.

Postconditions

- **Success Guarantee**
 - Code parsed to respective SDK.
 - Errors reported.
 - Circuit stored for validation/execution.

Main Success Scenario

1. User submits code to API.
2. System authenticates and validates format.
3. API forwards to IR Core.
4. IR Core detects dialect.
5. Converted to respective SDK.
6. Syntax validated.
7. Confirmation with metadata returned.

Extensions

- **4a. Unsupported Language**
 - 4a1. Dialect unidentified.
 - 4a2. “Unsupported language” error.
- **5a. Parsing Error**
 - 5a1. Syntax/semantic issue.
 - 5a2. Error logged.
 - 5a3. Details and line references returned.
- **7a. IR Validation Fails**
 - 7a1. Invalid structure.
 - 7a2. Job marked “invalid IR”.
 - 7a3. Hints provided.

Special Requirements

- Report detailed errors.
- Log activities.

Technology Requirements

- **Parser:** AST-based Python.
 - **Database:** PostgreSQL metadata.
 - **Logging:** Prometheus/Grafana.
-

2.1.2.3 Use Case 3: Execute Quantum Circuit on Backend

Use Case ID: UC-03

Use Case Name: Execute Quantum Circuit on Backend

Actors

- **Primary:** System

Stakeholders and Interests

- **User:** Expects accurate, timely circuit execution.
- **System Administrator:** Seeks efficient resource use.
- **Developers:** Require modular backend execution.

Preconditions

- Job is queued in RabbitMQ.
- Compute resources available in Kubernetes.
- Backend simulator (Qiskit Aer/Cirq/PennyLane) operational.
- OpenQASM 3.0 IR is valid.

Postconditions

- **Success Guarantee**
 - Job status updated to “completed” or “failed”.
 - Results stored in Amazon S3 and linked to job.
 - Resources released for reuse.
 - Circuit executed, results stored securely, resources released.

Main Success Scenario

1. Scheduler dequeues job from RabbitMQ.
2. Scheduler analyzes requirements (qubits, gates, type).
3. Scheduler estimates resources predictively.
4. Scheduler selects Kubernetes node pool (CPU/GPU/memory).
5. Scheduler creates Kubernetes Job specification.
6. Kubernetes provisions worker pod.
7. Pod retrieves job details and SDK formatted circuit.
8. Pod executes simulation.
9. Pod serializes and uploads results to S3.
10. Status updated to “completed”.

11. User notified of completion.
12. Pod terminated; resources released.

Extensions

- **4a. Insufficient Resources**
 - 4a1. No suitable nodes available.
 - 4a2. Job remains queued.
 - 4a3. System waits for resources.
 - 4a4. Retry after timeout.
- **9a. Simulation Fails**
 - 9a1. Runtime error occurs.
 - 9a2. Pod logs error.
 - 9a3. Status updated to “failed”.
 - 9a4. User notified.
 - 9a5. Resources released.

Special Requirements

- Support GPU acceleration if supported by respective SDK operation.
- Provide fault tolerance with retries.
- Isolate resources between jobs.
- Stream logs in real-time.

Technology Requirements

- **Simulators:** Qiskit Aer, Cirq, PennyLane.
- **Orchestration:** Kubernetes.
- **Queue:** RabbitMQ.
- **Database:** PostgreSQL.
- **Storage:** Amazon S3.
- **Monitoring:** Prometheus.

2.1.2.4 Use Case 4: Retrieve Simulation Results

Use Case ID: UC-04

Use Case Name: Retrieve Simulation Results

Actors

- **Primary:** User

Stakeholders and Interests

- **User:** Seeks efficient access to accurate simulation results.
- **System Administrator:** Ensures secure retrieval and system integrity.
- **Organization:** Requires reliable delivery and billing.

Preconditions

- Job status is “completed”.
- Results are stored in Amazon S3.
- User has a valid job ID.
- User is authenticated and authorized.

Postconditions

- **Success Guarantee**
 - User receives complete simulation results.
 - System logs retrieval and updates usage metrics.
 - Results remain accessible within retention period.

Main Success Scenario

1. User requests for results with specific job ID.
2. System authenticates the user.
3. System verifies job ownership.
4. Job manager checks status in PostgreSQL.
5. System confirms “completed” status.

6. System retrieves result metadata.
7. System fetches data from Amazon S3.
8. System deserializes data by simulation type.
9. System constructs result object (e.g., statevector, counts).
10. System logs usage for billing.
11. System returns formatted results.

Extensions

- **5a. Job Not Completed**
 - 5a1. Status is not “completed”.
 - 5a2. System returns “Results not available” error with status.
- **7a. Results Not Found in S3**
 - 7a1. Retrieval fails.
 - 7a2. System logs error.
 - 7a3. System returns “Results unavailable” error.
 - 7a4. System notifies administrators.

Special Requirements

- Cache results for repeated queries.
- Clean up results after retention period.
- Secure data transfer (HTTPS/SSL).
- Scale for large files and high demand.

Technology Requirements

- **Storage:** Amazon S3, Google Cloud Storage, or Azure Blob.
- **Data Format:** JSON, HDF5, or binary.
- **API:** REST or GraphQL.
- **Deserialization:** Type-specific (e.g., statevector).

2.1.2.5 Use Case 5: Manage Resource Allocation

Use Case ID: UC-05

Use Case Name: Manage Resource Allocation

Actors

- **Primary:** System

Stakeholders and Interests

- **User:** Desires fair, efficient resource access.
- **System Administrator:** Needs optimized utilization and workload balance.
- **Cloud Provider:** Seeks minimal idle costs.

Preconditions

- Job queued and pending.
- User authenticated and authorized.
- Prometheus monitoring operational.
- Quota policies configured.

Postconditions

- **Success Guarantee**
 - Resources allocated successfully.
 - Usage updated in database.
 - Cluster utilization reflects changes.
 - Job receives resources without quota or overload violations.

Main Success Scenario

1. Job manager receives allocation request.
2. System retrieves requirements from circuit analysis.
3. System estimates CPU, memory, GPU needs.
4. System checks cluster availability via Prometheus.

5. System classifies job priority (high/medium/low).
6. System verifies user quota.
7. System selects Kubernetes node pool.
8. System reserves resources.
9. System updates tracking in database.
10. System confirms allocation.

Extensions

- **4a. Insufficient Resources**
 - 4a1. No capacity available.
 - 4a2. Job remains queued.
 - 4a3. System estimates wait time.
 - 4a4. User notified of delay.
- **8a. Quota Exceeded**
 - 8a1. Limit reached.
 - 8a2. Request rejected.
 - 8a3. Status set to “quota_exceeded”.
 - 8a4. User notified.

Special Requirements

- Support priority and GPU-aware allocation.
- Track resources in real-time.
- Enforce dynamic user quotas.

Technology Requirements

- **Scheduler:** Kubernetes.
- **Monitoring:** Prometheus.
- **Database:** PostgreSQL.
- **Queue:** RabbitMQ.

2.1.2.6 Use Case 6: Schedule Jobs

Use Case ID: UC-06

Use Case Name: Schedule Jobs

Actors

- **Primary:** System

Stakeholders and Interests

- **User:** Expects timely job execution without indefinite delays.
- **System Administrator:** Requires balanced resource utilization and minimal bottlenecks.
- **Organization:** Demands efficient throughput and cost optimization.

Preconditions

- Jobs are queued in RabbitMQ.
- Kubernetes cluster is operational with available nodes.
- Resource monitoring via Prometheus is active.
- Scheduling policies (priority, quotas) are configured.

Postconditions

- **Success Guarantee**
 - Jobs are dequeued and assigned to appropriate resources.
 - Scheduling metadata updated in database.
 - Job status transitioned to “scheduled”.
 - Resources provisioned without conflicts.
- **Minimal Guarantee**
 - Failed scheduling attempts logged.
 - Jobs remain queued for retry.

Main Success Scenario

1. Scheduler polls RabbitMQ for pending jobs.
2. Scheduler dequeues the next job based on priority.
3. Scheduler parses job requirements (qubits, gates, SDK type).
4. Scheduler queries Prometheus for current cluster utilization.
5. Scheduler applies scheduling algorithm to select eligible nodes.
6. Scheduler verifies user quota and compliance.
7. Scheduler generates Kubernetes pod specification.
8. Scheduler submits pod to Kubernetes API.
9. Kubernetes confirms pod creation.
10. Job status updated to “scheduled” in PostgreSQL.
11. Scheduler notifies job manager of assignment.

Extensions

- **3a. High-Priority Job Detected**
 - 3a1. Priority flag evaluated.
 - 3a2. Job promoted ahead in queue.
 - 3a3. Lower-priority jobs deferred.
- **5a. Resource Contention**
 - 5a1. All nodes over capacity threshold.
 - 5a2. Job re-queued with backoff.
 - 5a3. Estimated wait time calculated.
 - 5a4. User notified via status update.
- **6a. Quota Violation**
 - 6a1. User daily/weekly limit exceeded.
 - 6a2. Job rejected for scheduling.
 - 6a3. Status set to “quota_exceeded”.
 - 6a4. Detailed quota info returned.

- **8a. Kubernetes API Failure**

- 8a1. Submission rejected (e.g., network issue).
- 8a2. Retry with exponential backoff.
- 8a3. Failure logged and alerted.

Special Requirements

- Implement priority queuing (high/medium/low).
- Support dynamic scaling triggers.
- Enforce fair-share across users.
- Provide estimated start times.
- Handle preemption for critical jobs.

Technology Requirements

- **Queue:** RabbitMQ with priority queues.
 - **Orchestration:** Kubernetes scheduler extensions.
 - **Monitoring:** Prometheus for utilization metrics.
 - **Database:** PostgreSQL for job metadata.
-

2.1.2.7 Use Case 7: Monitor Job Status

Use Case ID: UC-07

Use Case Name: Monitor Job Status

Actors

- **Primary:** User

Stakeholders and Interests

- **User:** Seeks real-time job execution updates.
- **System Administrator:** Requires visibility into the job pipeline.

Preconditions

- User has submitted at least one job.
- User has a valid job ID.
- User is authenticated.
- Monitoring system is operational.

Postconditions

- **Success Guarantee**
 - User receives the current job status.
 - Status information is accurate and current.
 - User can act on the status.

Main Success Scenario

1. User requests job status using the unique ID.
2. System authenticates the user and verifies job ownership.
3. Job management module retrieves the job record from the database.
4. System gathers current status and metrics from monitoring services.
5. System determines job state details:
 - Queued: Returns queue position and estimated wait time.
 - Running: Returns progress percentage, elapsed time, estimated completion.
 - Completed: Returns completion time and result availability.
 - Failed: Returns error details and reason.
6. System compiles status information:
 - Current status (queued/running/completed/failed/cancelled).
 - Job metadata (submission time, circuit size, backend).
 - Resource allocation (CPU/memory/GPU).
 - Execution metrics (runtime, queue time).
7. System returns structured status data to the user.

Real-Time Status Updates

1. User establishes a WebSocket connection.
2. System authenticates the connection.
3. System subscribes the user to the job status channel.
4. On status change:
 - System publishes update via RabbitMQ.
 - WebSocket service pushes the update to the user.

Batch Status Query

1. User requests status for up to 50 job IDs.
2. System authenticates the user.
3. System validates job ownership.
4. System queries the database for records.
5. System retrieves status for each job.
6. System returns a compiled status array.

Extensions

- **2a. Authentication Fails**
 - 2a1. System rejects credentials.
 - 2a2. System returns an authentication error.
- **3a. Job ID Not Found**
 - 3a1. Database query yields no results.
 - 3a2. System returns a “Job not found” error.
- **4a. Monitoring System Unavailable**
 - 4a1. Metrics fetch from Prometheus fails.
 - 4a2. System returns basic status without metrics.
 - 4a3. System logs the failure.
 - 4a4. Use case continues with reduced information.

- **Batch Query Extensions**
 - **5a. One or More Jobs Not Found:**
 - * 5a1. Invalid job IDs detected.
 - * 5a2. System returns status for valid jobs.
 - * 5a3. System lists invalid IDs.
 - **5b. Unauthorized Job Access:**
 - * 5b1. Unauthorized jobs identified.
 - * 5b2. System excludes them from results.
 - * 5b3. System logs the attempt.
- **6a. WebSocket Connection Failed**
 - 6a1. Connection establishment fails.
 - 6a2. System falls back to polling.
 - 6a3. User is notified to poll periodically (e.g., every 10 seconds).

Special Requirements

- Support batch queries up to 50 jobs.
- Provide real-time updates via WebSocket.
- Track historical status transitions.
- Allow polling with minimum 5-second intervals.
- Track detailed progress for running jobs.
- Show queue position for queued jobs.
- Calculate estimated completion times.

Technology Requirements

- **Database:** PostgreSQL (indexed status column).
- **Message Queue:** RabbitMQ for workflows and updates.
- **Monitoring:** Prometheus for metrics.
- **Backend:** Python REST API (FastAPI/Flask).

- **Real-Time:** WebSocket (Socket.IO/FastAPI).
 - **Authentication:** JWT or OAuth 2.0.
 - **Deployment:** Docker/Kubernetes.
 - **Logging:** ELK Stack or Grafana Loki.
 - **Message Broker:** Pub/Sub for broadcasting.
-

2.1.2.8 Use Case 8: Handle Job Failure and Retry

Use Case ID: UC-08

Use Case Name: Handle Job Failure and Retry

Actors

- **Primary:** System

Stakeholders and Interests

- **User:** Prefers automatic retries for failures.
- **System Administrator:** Needs visibility into failures.
- **System:** Ensures reliability without infinite loops.

Preconditions

- Job execution has failed.
- Monitoring and logging active.
- Retry policies configured.

Postconditions

- **Success Guarantee**
 - Job retried successfully or marked failed.
 - Logs and metrics stored.
 - Failed pod resources released.
 - Transient failures retried; persistent ones notified.

Main Success Scenario

1. Monitoring detects failure.
2. System captures details (logs, trace).
3. System classifies failure (transient/permanent).
4. System checks retry count.
5. System logs failure.
6. Status updated to “retrying”.
7. Failed pod cleaned up.
8. Job re-queued with incremented count.
9. Backoff period observed.
10. Job re-executed.
11. If successful, completes normally.

Extensions

- **3a. Permanent Failure**
 - 3a1. Non-transient error.
 - 3a2. Job marked failed.
 - 3a3. Detailed logs stored.
 - 3a4. User and admin notified.
- **5a. Max Retries Exceeded**
 - 5a1. Specific number of retries attempted.
 - 5a2. Status set to “failed”.
 - 5a3. User notified.
 - 5a4. Admin alerted.

Special Requirements

- Log errors and events in detail.
- Integrate alerting (Prometheus, email).

Technology Requirements

- **Orchestration:** Kubernetes.
 - **Monitoring:** Prometheus.
 - **Logging:** ELK or Grafana Loki.
 - **Database:** PostgreSQL.
 - **Queue:** RabbitMQ.
 - **Notifications:** Email/webhook.
-

2.1.2.9 Use Case 9: Cancel Running Job

Use Case ID: UC-09

Use Case Name: Cancel Running Job

Actors

- **Primary:** User
- **Secondary:** System

Stakeholders and Interests

- **User:** Cancels unneeded jobs.
- **System Administrator:** Ensures cleanup.
- **System:** Handles graceful termination.

Preconditions

- User authenticated.
- Job queued or running.

Postconditions

- **Success Guarantee**
 - Job cancelled.
 - Resources released.

- Status “cancelled”.
- Confirmation sent.

Main Success Scenario

1. User sends cancel request with ID.
2. System authenticates user.
3. System verifies ownership.
4. System checks status.
5. Status set to “cancelling”.
6. Queued: Remove from RabbitMQ.
7. Running: Signal Kubernetes pod.
8. Cancellation timestamp recorded.
9. Resources released.
10. Status “cancelled”.
11. User notified.

Extensions

- **5a. Not Cancellable**
 - 5a1. Status “completed”/“failed”.
 - 5a2. “Cannot cancel” error.
- **8a. Pod Unresponsive**
 - 8a1. Shutdown timeout.
 - 8a2. Force termination.

Special Requirements

- 30-second graceful shutdown.
- Save checkpoints for long jobs.
- Immediate for queued jobs.
- Ensure cleanup.

Technology Requirements

- **Backend:** FastAPI endpoint.
 - **Queue:** RabbitMQ removal.
 - **Orchestration:** Kubernetes API.
 - **Database:** PostgreSQL update.
 - **Logging:** Centralized.
-

2.1.2.10 Use Case 10: Monitor System Health and Performance

Use Case ID: UC-10

Use Case Name: Monitor System Health and Performance

Actors

- **Primary:** System Administrator
- **Secondary:** System

Stakeholders and Interests

- **System Administrator:** Needs comprehensive health metrics.
- **User:** Benefits from reliability.
- **Organization:** Requires proactive monitoring.

Preconditions

- Prometheus and Grafana operational.
- Metrics exporters configured.

Postconditions

- **Success Guarantee**
 - Administrator views real-time health.
 - Trends analyzed.
 - Alerts generated for issues.

Main Success Scenario

1. Administrator accesses Grafana dashboard.
2. System authenticates administrator.
3. Grafana queries Prometheus for metrics.
4. Prometheus aggregates data.
5. Dashboard displays KPIs (CPU, memory, queue depth).
6. Real-time graphs and trends shown.
7. Node and user metrics displayed.
8. Administrator reviews data.
9. Dashboard updates continuously.

Extensions

- **5a. High Queue Depth**

- 5a1. Threshold exceeded.
- 5a2. Alert generated.
- 5a3. Notification sent.
- 5a4. Alert shown on dashboard.

- **5b. Resource Saturation**

- 5b1. Utilization >90%.
- 5b2. Scaling alert triggered.
- 5b3. Recommendations provided.

- **2a. Authentication Failure**

- 2a1. Invalid credentials.
- 2a2. Access denied; attempt logged.
- 2a3. Retry prompted.

Special Requirements

- Refresh rate: 10–30 seconds.
- Retention: 30 days minimum.
- Custom alerts and thresholds.
- Responsive interface (mobile/tablet).
- Email/Slack notifications.

Technology Requirements

- **Monitoring:** Prometheus/Grafana.
 - **Storage:** Prometheus TSDB.
 - **Alerting:** Alertmanager/Grafana.
 - **Integration:** Kubernetes exporters.
 - **Authentication:** OAuth2/LDAP.
-

2.1.2.11 Use Case 11: View Job History and Analytics

Use Case ID: UC-11

Use Case Name: View Job History and Analytics

Actors

- **Primary:** User
- **Secondary:** System (API, Database)

Stakeholders and Interests

- **User:** Reviews past executions and patterns.
- **System Administrator:** Uses data for planning.
- **Organization:** Analyzes for billing/optimization.

Preconditions

- User authenticated.
- Job records in PostgreSQL.

Postconditions

- **Success Guarantee**
 - User receives matching history.
 - Analytics summary accurate.
 - Results paginated.

Main Success Scenario

1. User requests history via API/interface.
2. System authenticates user.
3. User specifies parameters (date, status, backend).
4. System queries PostgreSQL.
5. System filters to user jobs.
6. System retrieves metadata.
7. System computes statistics (jobs, success rate, runtime).
8. System formats response with list and summary.
9. Results paginated.
10. User views data.

Extensions

- **5a. No Jobs Found**
 - 5a1. Empty results.
 - 5a2. Empty list with message.
 - 5a3. Suggest filter changes.

- **5b. Too Many Results**
 - 5b1. >1000 records.
 - 5b2. Limit to 1000.
 - 5b3. Provide pagination token.

Special Requirements

- Support pagination, filtering, sorting.
- Visualize (charts).
- Real-time for ongoing jobs.

Technology Requirements

- **Backend:** Python API (FastAPI/Flask).
 - **Database:** PostgreSQL (indexed).
 - **Analytics:** SQL or module.
-

2.1.2.12 Use Case 12: Manage User Session

Use Case ID: UC-12

Use Case Name: Manage User Session

Actors

- **Primary:** User
- **Secondary:** Authentication Service, System

Stakeholders and Interests

- **User:** Desires secure, seamless access.
- **System Administrator:** Needs secure management.
- **Security Team:** Prevents hijacking.

Preconditions

- User has valid credentials.
- Authentication service operational.
- Session storage available.

Postconditions

- **Success Guarantee**
 - User authenticated with active session.
 - Token generated and stored securely.
 - Protected resources accessible.
 - Session tracked.

Main Success Scenario (Login)

1. User submits credentials.
2. System validates format.
3. Authentication verifies against database.
4. System checks account status.
5. Session stored in Redis (with expiration).
6. Login timestamp and IP logged.
7. Token returned.
8. User accesses endpoints.
9. Activity tracked.

Extensions

- **3a. Invalid Credentials**
 - 3a1. Incorrect username/password.
 - 3a2. Failed counter incremented.
 - 3a3. “Invalid credentials” error.
 - 3a4. Lock after 5 attempts.

- **4a. Account Suspended/Locked**

- 4a1. Status detected.
- 4a2. Appropriate error returned.
- 4a3. Event logged.
- 4a4. Contact admin advised.

Validation Failures

- **5a. Expired:** 401 error; re-authenticate.
- **5b. Invalid:** Log incident; 401 error.
- **5c. Revoked:** “Session terminated” error; re-authenticate.

Logout Scenario

1. User sends logout with token.
2. Token validated.
3. Session removed.
4. Token blacklisted.
5. Logout logged.
6. Confirmation returned.

Session Timeout and Cleanup

1. Periodic cleanup (every 5 minutes).
2. Expired sessions removed.
3. Statistics logged.
4. Inactive (30 minutes) expired.

Special Requirements

- Token expiration: 24 hours.
- Inactivity: 30 minutes.
- Token refresh support.
- Secure storage (httpOnly/headers).

- CSRF protection.
- Rate limit: 5/15 minutes.
- Multiple sessions allowed.
- Activity logging.

Technology Requirements

- **Authentication:** JWT (RS256).
 - **Database:** PostgreSQL.
 - **Hashing:** bcrypt/Argon2.
 - **Framework:** FastAPI OAuth2.
 - **Logging:** ELK.
-

2.1.3 Event-Response Table

The following table presents a comprehensive mapping of system events to their corresponding responses and associated use cases. (see Table 2.1)

Table 2.1: Event-Response Table

Event	Response	Use Case
User submits job	Authenticate, validate, translate to IR, enqueue, return ID	UC-01
User queries status	Authenticate, retrieve from DB, return state	UC-07
User requests results	Authenticate, verify completion, fetch from S3, return	UC-04
Job dequeued	Analyze, allocate resources, create K8s job	UC-06, UC-05
Circuit for translation	Parse, translate gates, validate, return OpenQASM	UC-02
Resources requested	Estimate, check availability/quota, allocate	UC-05

Continued on next page

Continued from previous page

Event	Response	Use Case
Execution fails	Capture error, retry if eligible, else fail	UC-08
Pod crashes	Detect, restart, update status, retry	UC-08
Simulation completes	Serialize, upload S3, update status, notify	UC-03
Quota exceeded	Reject, return error, notify	UC-01, UC-05
High queue depth	Alert, notify admins, dashboard display	UC-10
Resource saturation	Scaling alert, log, notify	UC-10
Authentication fails	Reject, return error, log	UC-12
Invalid syntax	Validate, return error details	UC-01, UC-02
Unsupported feature	Detect, return error	UC-02
S3 fails	Retry, log, fail if exhausted	UC-03, UC-04
Metrics unavailable	Use cache, continue degraded	UC-05, UC-10
Max retries	Mark failed, notify, alert	UC-08
Translation fails	Log, fail status, notify	UC-01, UC-02
Pod creation fails	Retry, re-queue	UC-06
Timeout	Terminate, checkpoint, update	UC-03, UC-08
Dashboard access	Authenticate, query, visualize	UC-10
Threshold exceeded	Alert, notify, log	UC-10
Priority set	Classify, adjust queue, allocate	UC-05
Checkpoint saved	Store S3, log, continue	UC-03, UC-08
Node unavailable	Detect, reschedule, update pool	UC-05, UC-06
Job cancelled	Update status, terminate, cleanup	UC-09
History requested	Authenticate, query, analytics, return	UC-11
Queued cancel	Remove queue, update, notify	UC-09
Running cancel	Signal, checkpoint, terminate, notify	UC-09
Pagination	Return next page with token	UC-11

2.2 Functional Requirements

This section describes the functional requirements of the system, derived from the use cases (UC-01 to UC-12).

2.2.1 IR Core

Following are the requirements for the IR Core module:

1. **FR1-IC:** The system shall make an AST from OpenQASM 3.0.
2. **FR2-IC:** The system shall provide errors for constructs not supported in the scope of OpenQASM 3.0.
3. **FR3-IC:** The system shall detect and report syntax/semantic errors during parsing, including line references.
4. **FR4-IC:** The system shall translate the parsed AST to SDK-specific circuits (Qiskit, Cirq, PennyLane).

2.2.2 Backend Adapters

Following are the requirements for the Backend Adapters module:

1. **FR1-BA:** The system shall provide an adapter to execute the translated OpenQASM 3.0 IR on a Qiskit backend.
2. **FR2-BA:** The system shall provide an adapter to execute the translated OpenQASM 3.0 IR on a Cirq backend.
3. **FR3-BA:** The system shall provide an adapter to execute the translated OpenQASM 3.0 IR on a PennyLane backend.
4. **FR4-BA:** The system shall correctly retrieve and format execution results (e.g., measurement outcomes, state vectors) from each supported backend framework.
5. **FR5-BA:** The system shall support GPU acceleration where available in the backend SDKs.

2.2.3 Job Manager

Following are the requirements for the Job Manager module:

1. **FR1-JM:** The system shall allocate computational resources (e.g., CPU, memory, GPU) required for a quantum simulation job.

2. **FR2-JM:** The system shall orchestrate the execution of jobs across available compute nodes.
3. **FR3-JM:** The system shall monitor the status of active jobs (e.g., pending, running, completed, failed) and update in real-time.
4. **FR4-JM:** The system shall queue incoming jobs when all compute nodes are occupied and execute them as resources become available.
5. **FR5-JM:** The system shall handle job failures by classifying errors, retrying transient issues (up to a max count), and marking persistent failures.
6. **FR6-JM:** The system shall support job cancellation, including queue removal or pod termination with graceful shutdown.

2.2.4 API Layer

Following are the requirements for the API Layer module:

1. **FR1-API:** The system shall provide a pip-installable Python package for users to interact with the QSim platform.
2. **FR2-API:** The API shall expose functions for authenticating and establishing a user session, including token management and expiration.
3. **FR3-API:** The API shall allow users to submit quantum code as a job for execution, including validation and queuing.
4. **FR4-API:** The API shall provide functions to check the status of a submitted job, supporting single/batch queries and real-time WebSocket updates.
5. **FR5-API:** The API shall allow users to retrieve the results of a completed job, including deserialization by type.
6. **FR6-API:** The API shall support querying job history and analytics, with filtering, pagination, and summary statistics.

2.3 Non-Functional Requirements

This section specifies nonfunctional requirements of QSim, derived from the special requirements, preconditions/postconditions, and extensions in the use cases (UC-01 to UC-12).

2.3.1 Reliability

The following requirements ensure robust error handling and fault tolerance, aligning with failure scenarios in UC-01, UC-02, UC-03, UC-08, and UC-09.

- **REL-1:** The system shall define a failure as any unhandled exception that causes the Job Manager to crash or a valid quantum job to terminate incorrectly.
- **REL-2:** In the event of a backend adapter failure (e.g., loss of connection to a framework's simulation service), the Job Manager shall log the error, mark the affected job as 'failed', and continue to process other jobs in the queue without crashing.
- **REL-3:** The IR Core shall gracefully handle and report syntax errors in user-submitted OpenQASM 3.0 code without crashing the system.
- **REL-4:** The system shall automatically retry transient failures (e.g., resource unavailability) up to 3 times with exponential backoff before marking a job as failed.

2.3.2 Usability

The following requirements focus on intuitive interfaces and clear feedback, supporting user interactions in UC-01, UC-04, UC-07, UC-09, UC-11, and UC-12.

- **USE-1:** A new user with experience in Python and at least one of the target quantum frameworks (Qiskit, Cirq, PennyLane) shall be able to submit their first successful job through the Python API within 30 minutes, using the provided documentation.
- **USE-2:** All error messages returned by the API shall be human-readable and provide a clear indication of the problem (e.g., "Authentication failed: Invalid API key", "Qiskit backend not available").
- **USE-3:** The Python package shall provide clear and concise function names and docstrings that align with common practices in the scientific Python ecosystem.
- **USE-4:** The system shall support real-time status updates via WebSocket for running jobs, with fallback to polling at 5-second intervals.

2.3.3 Performance

The following requirements address scalability and efficiency, drawn from special requirements in UC-01, UC-03, UC-05, UC-07, and UC-10.

- **PERF-1:** The IR Core translation process shall add no more than 5% overhead to the total job execution time for circuits.
- **PERF-2:** The API response time for submitting a new job shall be less than 500 milliseconds under a load of 20 concurrent requests.
- **PERF-3:** The Job Manager shall be capable of queuing and managing at least 100 jobs simultaneously without significant degradation in scheduling performance.
- **PERF-4:** The end-to-end simulation time (from API submission to result retrieval) for a standard 10-qubit Bell state preparation and measurement circuit shall not be more than 15% longer than running the same circuit directly in the native framework (Qiskit, Cirq, or PennyLane).
- **PERF-5:** The system shall support concurrent job submissions from up to 50 users without queue overflow or API latency exceeding 1 second.

CHAPTER 3

SYSTEM OVERVIEW

This chapter provides a comprehensive overview of the QSim platform, detailing its core functionalities for quantum circuit simulation, the contextual challenges it addresses in the quantum computing landscape, and the high-level design principles guiding its architecture.

3.1 Architectural Design

This section details the microservice-based architecture of the QSim platform, designed for scalable quantum circuit simulation. The workflow is logically divided into three primary stages: job submission, asynchronous execution, and data persistence. The platform exposes its capabilities to end users through the **FastQSim SDK** (`fastqsim` Python package), which wraps all REST communication with the FastQubit API and provides a high-level, backend-agnostic interface for submitting and managing quantum simulation jobs.

3.1.1 Environment Setup and Job Submission

The process is initiated by a user request to set up a working environment and submit a quantum circuit for simulation.

- **QCanvas:** The primary user-facing interface through which a user begins by sending a Request for Resources. QCanvas acts as the front-end integrator and is responsible for injecting runtime environment variables — specifically `FASTQUBIT_ENDPOINT` and `FASTQUBIT_SESSION_TOKEN` — directly into the user’s session pod. This design allows the FastQSim SDK to be initialised inside the pod with no manual configuration:

```

1 import fastqsim
2 client = fastqsim.init() # reads FASTQUBIT_SESSION_TOKEN
   from environment

```

The `fastqsim.init()` call establishes a `FastQSimClient` instance configured with the pod-scoped session token (`fqp_... format`), a per-request timeout (default 300 seconds), and automatic retry logic for transient HTTP failures (default 3 retries). If the token is absent or invalid, an `AuthenticationError` is raised immediately; if the API endpoint cannot be reached, a `FastQSimConnectionError` is raised.

- **QSim:** Acts as the central orchestrator, receiving the initial resource request from QCanvas and coordinating the downstream provisioning workflow.
- **Environment Manager:** The service that QSim instructs to provision a dedicated computational environment for the user. It handles the full pod lifecycle including creation, readiness checks, token injection, and teardown. Sessions are represented in the SDK by the `Session` object, which tracks the pod through the following states:

`creating` → `provisioning` → `ready` → `active`
 → `idle` → `terminating` → `terminated`

Each session carries a unique `session_id` (UUID), a `ws_url` for the active Jupyter kernel WebSocket channel, a short-lived `ws_token` for authenticating that connection, and an `expires_at` hard TTL after which the pod is automatically reclaimed.

- **Simple Node Pool:** The pool of general-purpose Kubernetes nodes from which the Environment Manager allocates resources to create the user’s interactive environment. These nodes host the user-session pods that run the FastQSim SDK and accept circuit submissions.
- **QSim User API:** The asynchronous REST API within the provisioned environment through which the user submits their quantum circuit. Circuits must be expressed as **OpenQASM 3.0** strings — native circuit objects from Qiskit, Cirq, or PennyLane are not accepted directly. Submission is made via `client.run()`, which accepts the key parameters listed in Table 3.1.

When `asynchronous=True` (fire-and-forget mode), the API offloads the task and **immediately returns a Job object** containing a unique `job_id` (UUID) for tracking. The caller can then perform other work and retrieve results later via `client.wait(job_id, timeout=300)`. When `asynchronous=False` (synchronous/blocking mode), `client.run()` internally polls until the job reaches a terminal state before

Table 3.1: Key parameters of the `client.run()` API call

Parameter	Type	Description
circuit	str List[str]	OpenQASM 3.0 circuit string(s)
backend	str	Simulation backend: qiskit, cirq, or pennylane
device	str	Target device: cpu, gpu, or quantum
shots	int	Number of measurement repetitions (default: 1024)
simulation_type	str None	Simulation method (see Section 3.1.2)
seed	int None	Optional RNG seed for reproducible results
asynchronous	bool	If True, returns immediately; if False, blocks until completion
job_name	str None	Human-readable label for the job
tags	dict None	Key-value tags for filtering and reporting
metadata	dict None	Arbitrary metadata stored with the job

returning — best suited for interactive notebooks and short circuits.

For high-throughput workloads, `client.run_batch()` submits multiple circuits concurrently using a client-side thread pool. All circuits in the batch share the same backend, device, shots, and simulation type. The `max_parallel` parameter (default: 10) controls the level of client-side concurrency. Batch jobs are always asynchronous, and `client.wait()` is used to block on all of them collectively.

3.1.2 Asynchronous Job Processing and Execution

Once submitted, the job is processed through a decoupled, asynchronous pipeline before being executed on specialised hardware. A submitted job progresses through a well-defined lifecycle of states:

$$\text{CREATED} \rightarrow \text{QUEUED} \rightarrow \text{RUNNING} \rightarrow \begin{cases} \text{COMPLETED} \\ \text{FAILED} \\ \text{CANCELLED} \end{cases}$$

Terminal states (completed, failed, cancelled) are immutable. The SDK's Job Status enum captures all six possible states, and the `job.status.is_terminal` property can be used to check finality programmatically.

- **IR Parser:** The service that receives the submitted job and parses the circuit's Intermediate Representation (IR). The OpenQASM 3.0 circuit is validated and transformed into a standardised structure compatible with the target backend simulators. Malformed QASM or invalid parameters result in a `ValidationError`

(HTTP 422) at submission time, before the job enters the queue. The circuit profile (qubit count, depth, gate count, two-qubit gate count) is also extracted by an AST analyser at this stage and stored in `job.profile`.

- **RabbitMQ:** The message broker to which the parsed job is published. This queue acts as a resilient buffer that fully decouples the submission system from the execution backend. The timestamp at which the job is accepted and published is recorded in `job.queue_enqueued_at`. RabbitMQ ensures that bursts of job submissions do not overwhelm the execution layer, and that jobs are not lost if a worker pod becomes temporarily unavailable.
- **Kubernetes Job Scheduler:** Acts as a RabbitMQ consumer, fetching jobs from the queue when execution resources are available. The timestamp at which the scheduler claims a job and creates a worker pod is recorded in `job.queue_dequeued_at`. The scheduler is also responsible for adaptive Kubernetes resource sizing — the predicted CPU and memory allocation for the worker pod is accessible via `job.resource_estimate` (fields: `cpu_request`, `memory_request`, `cpu_limit`, `memory_limit`).
- **Quantum Node Pool:** The pool of specialised Kubernetes nodes to which the scheduler dispatches fetched jobs, optimised for quantum simulation workloads. The name of the specific Kubernetes pod that executed a job is available in `job.pod_name`. The moment the worker begins simulation is recorded in `job.execution_started_at`. The moment it completes and uploads any artifacts is recorded in `job.execution_finished_at`.
- **Simulation Backends and Types:** Each job targets one of three backends (`qiskit`, `cirq`, `pennylane`) and one of four simulation methods, summarised in Table 3.2. The `statevector` mode is the default when `simulation_type` is not specified.
- **Docker:** Each node in the Quantum Node Pool runs containerised simulators. When a job is assigned, the node fetches the required container image from a Docker registry, ensuring a consistent and reproducible execution environment for every simulation. An optional `seed` parameter can be supplied at submission time; when set, the same circuit produces identical measurement counts on every run across all backends.
- **Job Cancellation:** A queued or running job can be cancelled via `client.cancel(job_id)` or `job.cancel()`. Cancellation is idempotent — cancelling an already-terminal job is a no-op. Importantly, cancellation signals the platform to stop scheduling the job but does not forcefully terminate an already-running worker pod; if a worker is mid-execution it may finish, but any terminal state updates are dis-

carded once cancelled. A `JobCouldNotCancelError` is raised if the server rejects the request.

Table 3.2: Supported simulation types and their characteristics

simulation_type	Method	Memory	Gate Support
statevector	Full state-vector	$O(2^n)$	All gates (default)
density_matrix	Open-system / noisy	$O(4^n)$	All gates
mps	Matrix Product State	$O(N \cdot \chi^2)$	Low-entanglement circuits
chp	Stabilizer / Clifford	$O(N^2)$	Clifford gates only (H, CNOT, S, measure)

3.1.3 Results and Metrics Persistence

After the simulation is complete, output data and performance metrics are systematically collected and stored in appropriate data stores. The worker emits a `completed_at` timestamp once all result data has been finalised in persistent storage.

- **Bucket Storage:** The primary simulation output is uploaded to a scalable object storage solution (e.g., AWS S3, Google Cloud Storage). For standard-sized results, measurement counts (`result.counts`, a bitstring-to-count mapping such as `{'00': 512, '11': 512}`) and outcome probabilities (`result.probabilities`) are returned inline with the job response.

For large simulations — such as statevector outputs for high qubit counts — the result JSON exceeds the inline response limit and is instead stored as an **external artifact** in object storage. The `job.has_external_artifact` flag indicates when this has occurred. In that case, `job.result()` transparently fetches and merges the artifact via a presigned download URL valid until `job.result_url_expires_at`. The statevector — a list of complex amplitudes of length 2^n for an n -qubit circuit — is accessible via `job.get_statevector()`.

- **PostgreSQL:** Key metadata, lifecycle timestamps, and performance metrics are stored in a PostgreSQL relational database, vital for monitoring, billing, and providing users with complete job history. The full set of fields persisted per job includes:
 - **Identity & routing:** `job_id` (UUID), `session_id`, `integrator_id`, `end_user_id`, `pod_name`, `backend`, `device`, `simulation_type`, `shots`, `seed`
 - **Lifecycle timestamps** (all UTC): `created_at`, `queue_enqueued_at`, `queue_dequeued_at`, `execution_started_at`, `execution_finished_at`, `completed_at`, `updated_at`

- **Performance metrics:** `execution_time_seconds` (wall-clock), `cpu_seconds_total` (user + system CPU time), `peak_memory_mb` (peak RSS memory usage)
- **Billing telemetry:** `billing_cpu_millicore_seconds` (integrated CPU usage), `billing_memory_gb_seconds` (integrated memory usage), `billing_avg_memory_mb`, `billing_memory_headroom_mb`
- **Circuit profile:** `num_qubits`, `depth`, `gate count`, and `two-qubit gate count` (from AST analysis)
- **User metadata:** `job_name`, `tags` (key-value), `metadata` (arbitrary JSON), `options`
- **Error data:** `error_message` — the raw exception text from the worker pod, populated when `status = failed`
- **Result metadata:** backend name, simulation type, and engine version, stored in `result_metadata`

Job history can be queried programmatically via `client.search()`, which supports filtering by status, backend, and date range. It returns up to 500 records ordered by most-recently-updated first, enabling observability, debugging, and usage reporting without requiring direct database access.

3.1.4 Architectural Diagram

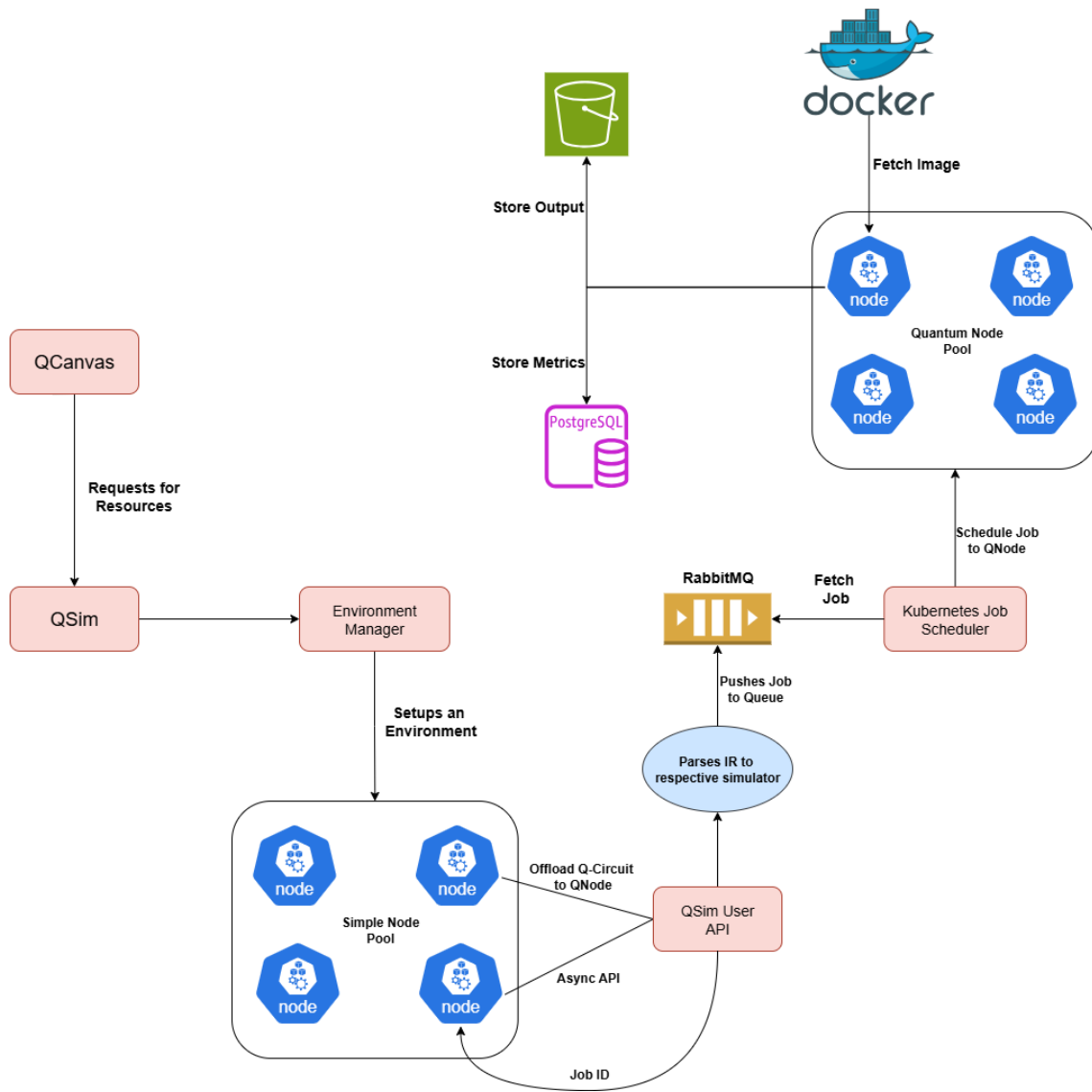


Figure 3.1: Architectural Diagram of the System

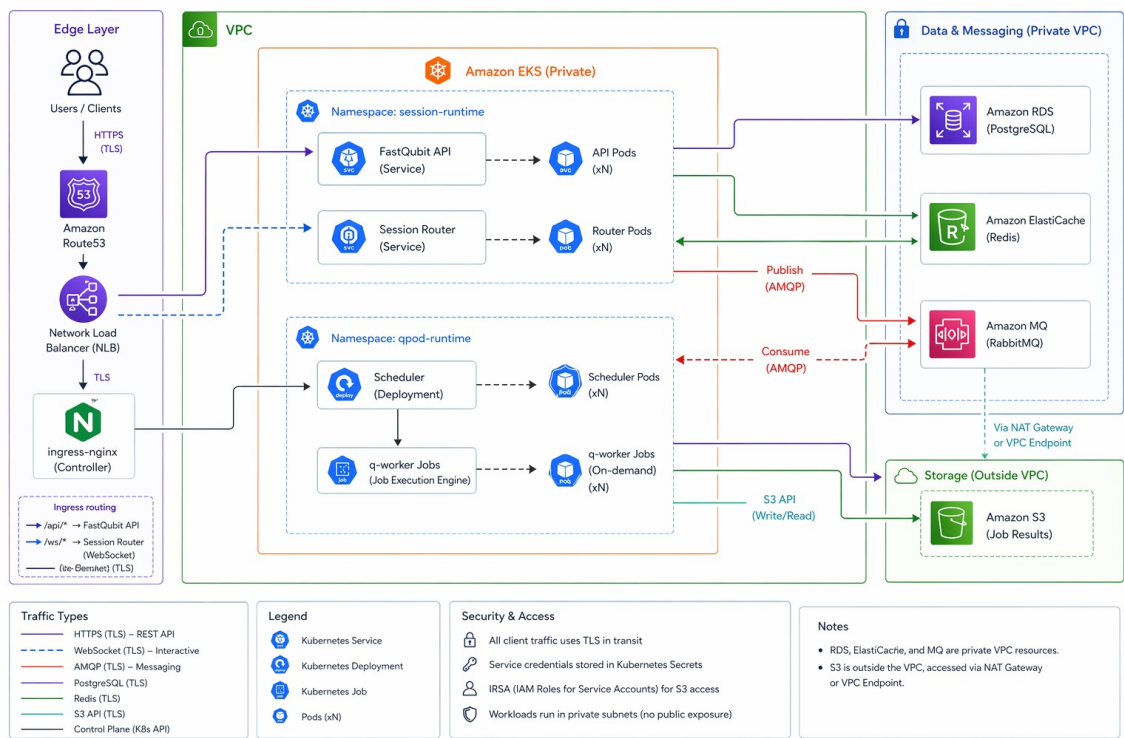


Figure 3.2: Cloud Deployment Architecture

3.2 Design Models

This section presents the key design models for the QSim system, highlighting the architectural structure, workflows, and interactions to provide a comprehensive view of the system's behavior and components.

3.2.1 Activity Diagram

The activity diagrams illustrate the high-level workflows and decision points across the system's core processes, including job submission, execution, and result retrieval, to model the dynamic flow of activities. (see Figures 3.3, 3.4)

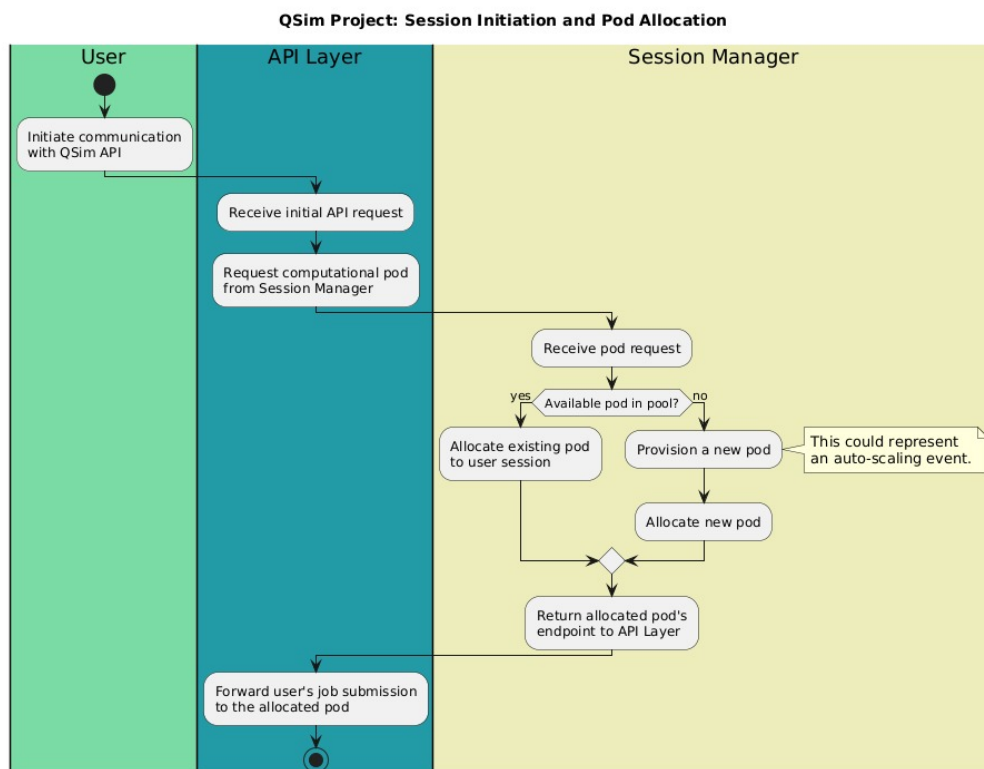


Figure 3.3: Activity Diagram-1

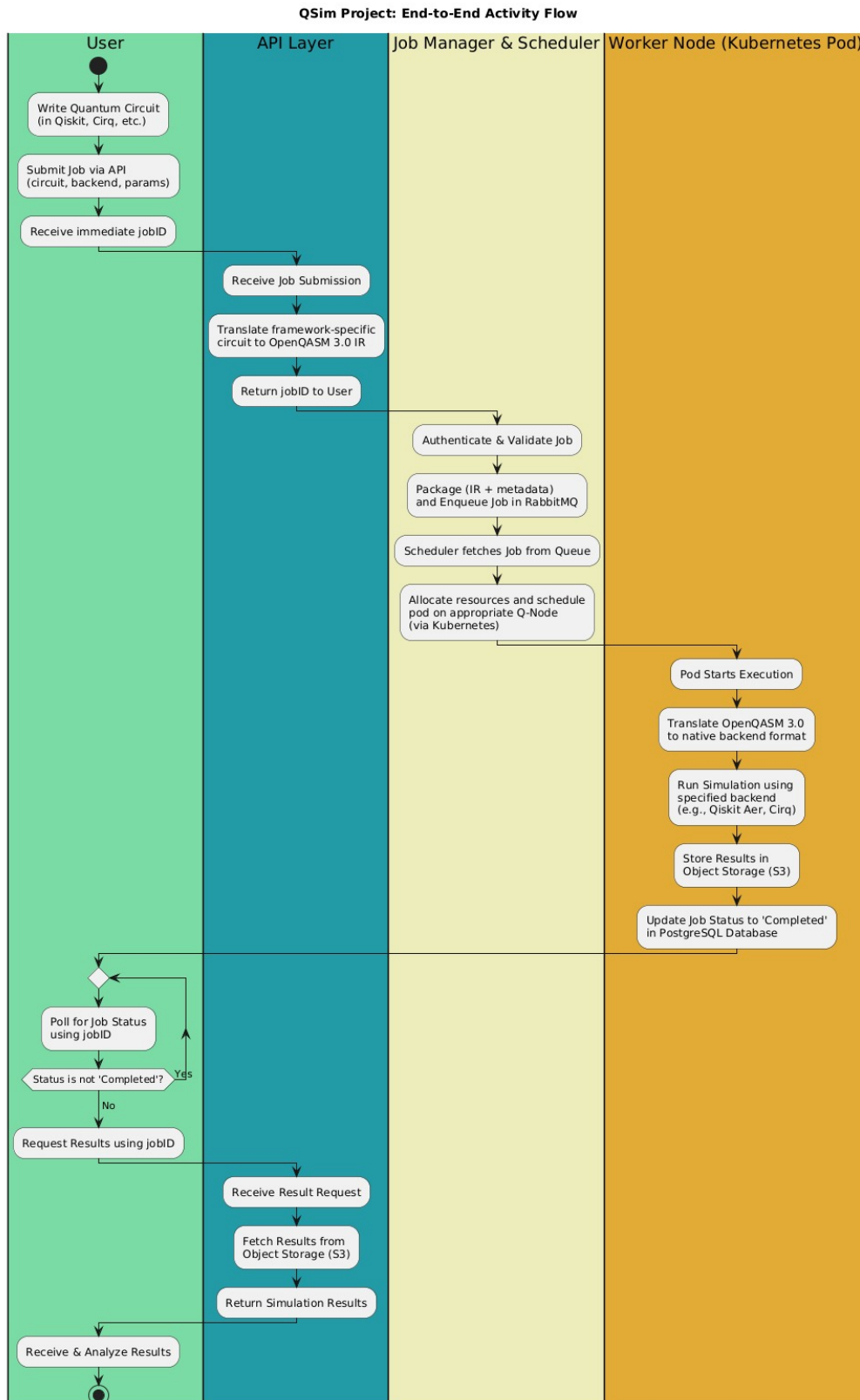


Figure 3.4: Activity Diagram-2

3.2.2 Class Diagram

The class diagram provides a static representation of the system’s core entities, their attributes, methods, and relationships, offering insight into the modular structure of components like the IR Core, Backend Adapters, and Job Manager. (see Figure 3.5)

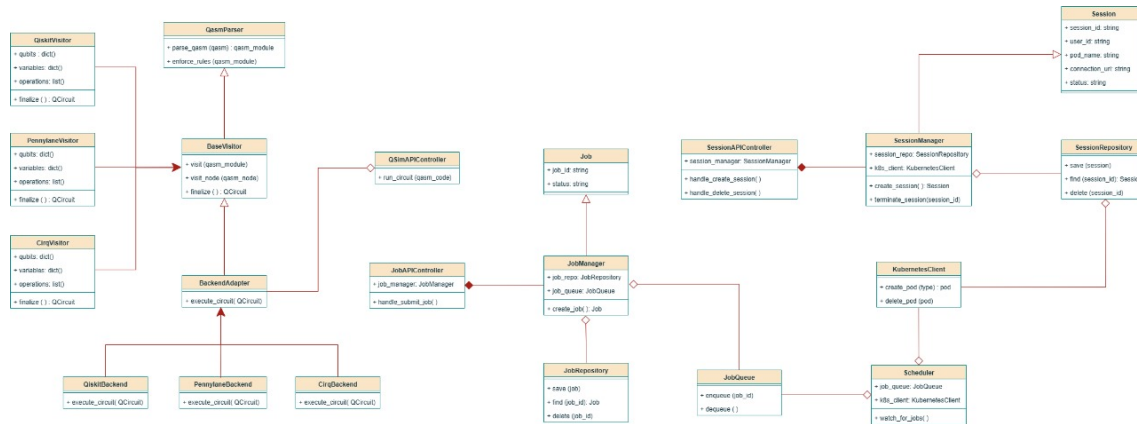


Figure 3.5: Class Diagram

3.2.3 Class-level Sequence Diagram

The sequence diagrams capture the dynamic collaborations between classes and objects during critical scenarios, such as user authentication, job orchestration, and failure handling, to demonstrate the temporal ordering of messages and operations. (see Figures 3.6, 3.7)

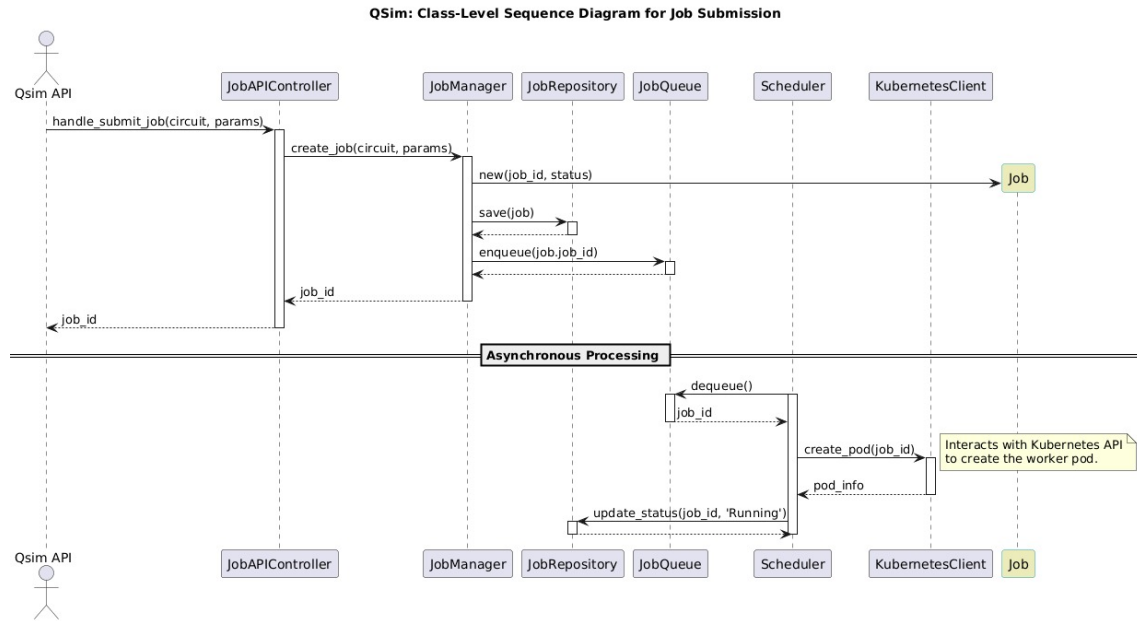


Figure 3.6: Sequence Diagram-1

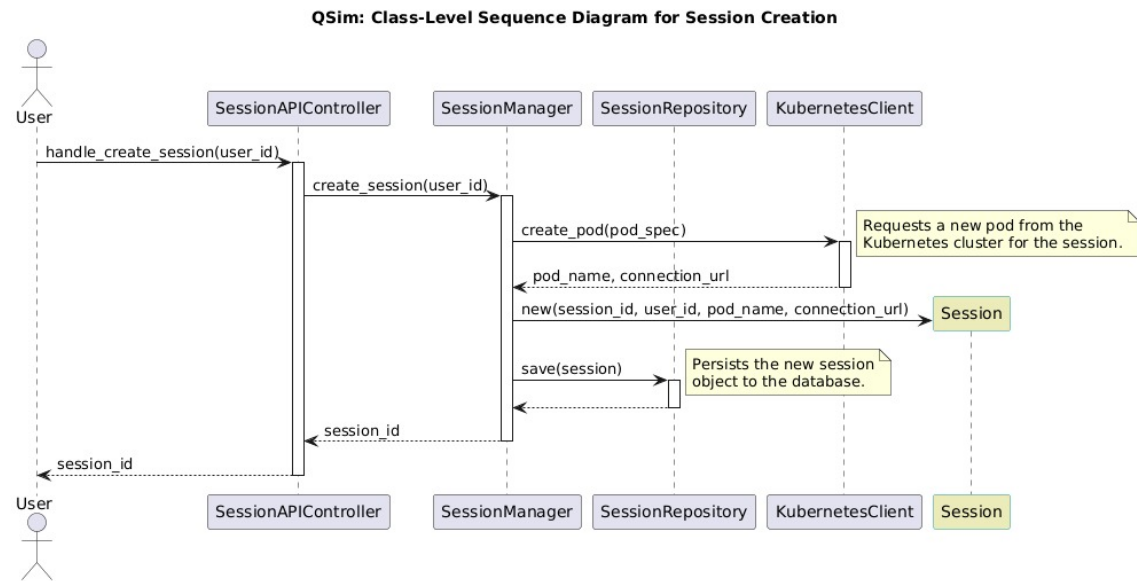


Figure 3.7: Sequence Diagram-2

3.3 Data Design

3.3.1 Overview

The system transforms user-submitted quantum circuits (in OpenQASM 3.0) into executable jobs and stores simulation results. Data flows from API input through processing

stages to persistent storage, optimized for fast queries, large-scale retention, and real-time monitoring.

3.3.2 Data Transformation Pipeline

1. User Input → Intermediate Representation (IR)

- OpenQASM 3.0 code parsed and validated into standardized IR format via IR Core.
- IR includes gate sequences, qubit configurations, measurements, and metadata (e.g., backend, shots).

2. IR → Job Specification

- IR translated to SDK-specific circuits (Qiskit, Cirq, PennyLane) and packaged with execution parameters (e.g., shots, seed).
- Job entity created with unique ID, user reference, status, and resource estimates (e.g., qubits, runtime).

3. Execution → Results

- Worker pods produce raw outputs (e.g., counts, statevectors) from backend simulators.
- Results serialized: metadata (e.g., status, timestamps) to database; large binaries (e.g., full statevectors) to object storage.

3.3.3 Major Data Entities

1. Jobs

- Attributes: ID, user ID, IR/circuit reference, backend, status (queued/running/completed/failed/cancelled), timestamps, resource requirements, retry count.
- Lifecycle tracked: submission → queued → running → completed/failed/cancelled.
- Stored in PostgreSQL for ACID-compliant updates and queries.

2. Simulation Results

- Attributes: job ID, outcomes (counts, probabilities), statevectors, performance metrics (e.g., execution time, fidelity).
- Compact summaries in database; full/large datasets (e.g., >1MB) in S3 with pre-signed URLs.

- Linked via job ID for retrieval and history analytics.

3. System Metrics

- Attributes: timestamps, resource usage (CPU/GPU/memory), queue depth, job latencies, error rates, node health.
- Time-series format for aggregation and alerting.
- Stored in PostgreSQL (or integrated with Prometheus TSDB) for dashboards and capacity planning.

4. User Sessions

- Attributes: user ID, JWT token, expiration, IP, activity logs.
- Tracked for authentication, quotas, and auditing.
- Stored in PostgreSQL with Redis for fast token validation (implied for scalability).

3.3.4 Data Storage Systems

PostgreSQL Database

- *Stores:* Job metadata, user sessions, result summaries, circuit IR snippets, metrics logs.
- *Purpose:* Structured queries (e.g., status filtering, history pagination), transactional integrity, indexing for performance.

Amazon S3 Object Storage

- *Stores:* Large simulation outputs (e.g., HDF5/JSON results, visualizations), full IR files, pod logs.
- *Purpose:* Durable, scalable storage for unstructured/binary data with versioning and lifecycle policies.

RabbitMQ Message Queue

- *Stores:* Serialized job specs (IR + metadata) for pending execution.
- *Purpose:* Reliable, asynchronous decoupling between submission and processing; supports prioritization.

Kubernetes/Docker Registry

- *Stores:* Container images for worker pods (e.g., SDK environments).
- *Purpose:* Efficient distribution and versioning of simulation runtimes to node pools.

3.3.5 Data Flow

Write Path:

User API → IR Core (parse/translate) → PostgreSQL (job metadata) → RabbitMQ (enqueue) → Job Manager/Scheduler (dequeue/allocate) → Kubernetes Pod (execute) → S3 (results) + PostgreSQL (update metadata/status)

Read Path:

User API → PostgreSQL (status/metadata/history) → S3 (large files via presigned URLs, if needed) → Deserialized results

3.3.6 Key Design Principles

- **Separation of Concerns:** Metadata and small payloads in relational DB; binaries and logs in object storage.
- **Scalability:** Horizontal scaling via Kubernetes partitioning; S3 for petabyte growth; RabbitMQ clustering for high throughput.
- **Consistency:** ACID transactions for job states (PostgreSQL); eventual consistency for results (S3 uploads with retries).
- **Security:** Data encrypted at rest (S3/PGSQL) and in transit (HTTPS); RBAC for access; audit logs for compliance.
- **Performance:** Caching (e.g., Redis for sessions/status); indexing on job/user IDs; async processing to minimize latency.

3.4 Domain Model

This section introduces the domain model for QSim, encapsulating the essential entities, attributes, and relationships that represent the quantum circuit simulation domain, derived from the system's use cases and requirements. (see Figure 3.8)

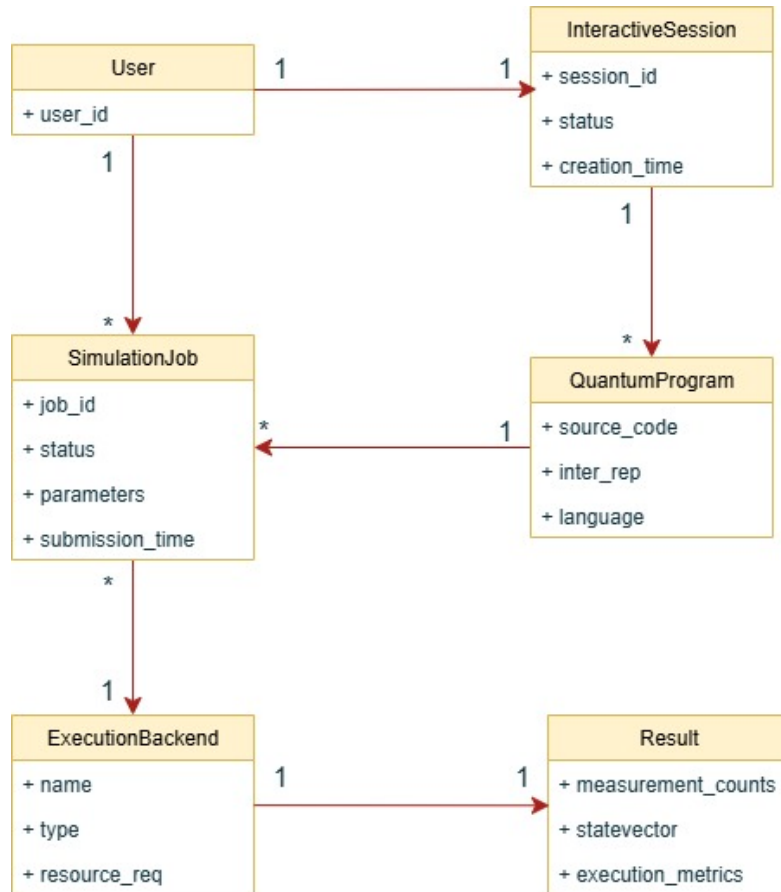


Figure 3.8: Domain Model

CHAPTER 4

IMPLEMENTATION AND TESTING

Here is a general description of the functionality, context, and design of our project.

4.1 Algorithm Design

This section describes the core algorithms implemented in the quantum simulation framework. The system follows a modular pipeline architecture consisting of parsing, circuit construction, and simulation execution phases.

4.1.1 Main Simulation Pipeline

4.1.1.1 QASM Simulation Orchestration

The primary algorithm coordinates the entire simulation workflow, managing the transformation from QASM input through to final results. It handles configuration processing, delegates to specialized visitors for circuit construction, and interfaces with backend simulators to execute quantum circuits. The algorithm includes comprehensive error handling and supports both shot-based and statevector simulation modes.

4.1.1.2 QASM Parser

This algorithm handles the parsing of OpenQASM 3 source code into an abstract syntax tree (AST). It supports both file paths and direct code strings, validates input, and optionally unrolls loops and enforces strict scoping rules. The parser serves as the entry point for all circuit definitions in the system.

Algorithm 1 Run QASM Simulation**Require:** *args*: Configuration object or dictionary containing:

- *qasm_input*: QASM source code string or file path
- *visitor_name*: Name of the visitor to use (e.g., 'qiskit', 'cirq')
- *backend_name*: (Optional) Name of the backend to use
- *shots*: Number of simulation shots

Ensure: *SimResult*: Object containing counts, probabilities, and metadata

```

1: function RUNQASM(args)
2:   if args is a dictionary then
3:     args  $\leftarrow$  CONVERTTORUNARGS(args)
4:   end if
5:                                      $\triangleright$  1. Parse QASM3 to AST
6:   qasm_input  $\leftarrow$  args.qasm_input
7:   if qasm_input is a Path then
8:     qasm_input  $\leftarrow$  READTEXT(qasm_input)
9:   end if
10:  module  $\leftarrow$  PARSEOPENQASM3(qasm_input)
11:                                      $\triangleright$  2. Visitor: Traverse AST and build circuit
12:  visitor  $\leftarrow$  GETVISITOR(args.visitor_name)
13:  VISITOR.VISIT(visitor, module)
14:  circuit  $\leftarrow$  VISITOR.FINALIZE(visitor)
15:                                      $\triangleright$  3. Backend: Run simulation
16:  backend_name  $\leftarrow$  args.backend_name or args.visitor_name
17:  backend  $\leftarrow$  GETBACKEND(backend_name)
18:  raw_result  $\leftarrow$  BACKEND.RUN(backend, circuit, args.shots)
19:  counts  $\leftarrow$  BACKEND.TOCOUNTS(backend, raw_result)
20:                                      $\triangleright$  4. Build Result
21:  n_qubits  $\leftarrow$  INFERQUBITCOUNT(circuit)
22:  metadata  $\leftarrow$  {'n_qubits' : n_qubits, 'visitor' : args.visitor_name, ... }
23:  if args.shots = 0 then
24:    probs  $\leftarrow$  raw_result.probs
25:  else
26:    probs  $\leftarrow$  None
27:  end if
28:  return SIMRESULT(counts, metadata, probs)
29:  Exception e
30:  if e is known QSim error then
31:    raise e
32:  else
33:    raise QSIMERROR("Simulation failed", e)
34:  end if
35: end function

```

Algorithm 2 Parse OpenQASM 3

```

1: function PARSEOPENQASM3(qasm_code, unroll, strict_scope)
2:   if qasm_code is Path then
3:     qasm_code  $\leftarrow$  READTEXT(qasm_code)
4:   end if
5:   if qasm_code is empty then
6:     raise ParseError
7:   end if
8:   module  $\leftarrow$  PYQASM.LOADS(qasm_code)
9:   ENFORCESCOPE(module, strict_scope)
10:  if unroll is True then
11:    MODULE.UNROLL
12:  end if
13:  return module
14: end function

```

4.1.2 Component Factory Methods**4.1.2.1 Visitor and Backend Factories**

These factory methods provide a unified interface for instantiating different quantum circuit visitors and simulation backends. They map string identifiers to concrete implementation classes, enabling flexible configuration and easy extension with new frameworks. The factories support multiple quantum computing libraries including Cirq, Qiskit, and others.

Algorithm 3 Factory Methods

```

1: function GETVISITOR(name)
2:   map  $\leftarrow$  { 'cirq' : CirqVisitor, 'qiskit' : QiskitVisitor, ... }
3:   cls  $\leftarrow$  map.GET(name.lower(), CirqVisitor)
4:   return CLS
5: end function

6: function GETBACKEND(name)
7:   map  $\leftarrow$  { 'cirq' : CirqBackend, 'qiskit' : QiskitBackend, ... }
8:   cls  $\leftarrow$  map.GET(name.lower(), CirqBackend)
9:   return CLS
10: end function

```

4.1.3 Circuit Construction**4.1.3.1 AST Visitor Pattern**

The visitor traversal algorithm implements the visitor design pattern to walk through the abstract syntax tree produced by the parser. It dynamically dispatches to specialized han-

pler methods based on node types, allowing different visitor implementations to construct circuits in their respective frameworks. This pattern enables clean separation between parsing and circuit generation logic.

Algorithm 4 Visitor Traversal (Overview)

```

1: function VISIT(module)
2:   for each node in module.statements do
3:     type  $\leftarrow$  TYPEOF(node)
4:     method  $\leftarrow$  "visit_" + type
5:     if HASMETHOD(self, method) then
6:       SELF.METHOD(node)
7:     else
8:       SELF.GENERIC_VISIT(node)
9:     end if
10:  end for
11: end function

```

4.1.4 Simulation Execution

4.1.4.1 Backend Interface

The backend simulation algorithm defines the abstract interface that all concrete backend implementations must satisfy. Each backend translates its framework-specific circuit representation into simulation results, providing both measurement counts and probability distributions. This abstraction allows the system to seamlessly switch between different quantum simulation engines.

Algorithm 5 Backend Simulation (Abstract)

```

1: function RUN(circuit, shots)
2:                                      $\triangleright$  Abstract interface implemented by specific backends
3:   result  $\leftarrow$  SIMULATE(circuit, shots)
4:   return {'counts' : ..., 'probs' : ...}
5: end function

```

4.1.5 Utility Functions

4.1.5.1 Qubit Count Inference

This utility algorithm determines the number of qubits in a circuit by checking various framework-specific properties and methods. It handles the differences between quantum computing libraries by attempting multiple access patterns in priority order, ensuring robust qubit counting across different circuit representations.

Algorithm 6 Infer Qubit Count

```

1: function INFERQUBITCOUNT(circuit)
2:   if circuit has property 'n_qubits' then
3:     return circuit.n_qubits
4:   else if circuit has property 'num_qubits' then
5:     return circuit.num_qubits
6:   else if circuit has method 'all_qubits' then
7:     return Length of circuit.all_qubits()
8:   else if circuit has property 'wires' then
9:     return Length of circuit.wires
10:  else
11:    return 1 ▷ Fallback
12:  end if
13: end function

```

4.2 Documentation and SDK Resources

To improve usability, accessibility, and developer adoption, the QSim framework provides comprehensive documentation resources, including interactive API documentation, SDK references, and a publicly distributed Python package. These resources enable developers and researchers to easily integrate QSim into their workflows, explore available APIs, and install the framework directly through the Python Package Index (PyPI).

4.2.1 Software Documentation and Package Distribution

The project documentation was designed to support both end users and developers by providing clear API references, usage examples, and installation instructions. The framework is distributed as an installable Python package and includes automatically generated API documentation using Swagger/OpenAPI standards.

4.2.2 Resource Access Links

The following resources are publicly available for installation, testing, and developer reference:

- **PyPI Package:**

<https://pypi.org/project/fastqsim/>

- **Swagger API Documentation:**

<https://aneeq-ahmed-malik.github.io/FastQubit/swagger.html>

- **SDK Documentation:**

<https://aneeq-ahmed-malik.github.io/QSim/>

Resource	Platform/Technology	Purpose
PyPI Package	Python Package Index (PyPI)	Provides public package distribution for easy installation and dependency management using <code>pip install fastqsim</code> .
Swagger API Documentation	OpenAPI / Swagger UI	Offers interactive REST API documentation for testing endpoints, viewing request-response schemas, and understanding backend services.
SDK Documentation	Static Documentation Website	Provides detailed SDK references, architecture explanations, setup instructions, and developer usage examples for integrating QSim into external applications.

Table 4.1: Documentation and SDK Resources for QSim

4.2.3 Installation Example

The QSim package can be installed directly from PyPI using the Python package manager:

```
pip install fastqsim
```

Listing 4.1: Installing QSim from PyPI

This packaging and documentation ecosystem improves reproducibility, simplifies integration into external systems, and enhances the accessibility of the framework for researchers, students, and developers working with OpenQASM 3.0 quantum circuit simulation.

4.3 External APIs/SDKs

The QSim framework integrates multiple quantum computing SDKs to provide comprehensive support for OpenQASM 3.0 circuit translation and simulation across different quantum platforms.

API and Version	Description	Purpose of Usage	API Endpoint/Function/Class Used
Core Quantum Computing SDKs			

Continued on next page

API and Version	Description	Purpose of Usage	API Endpoint/Function/Class Used
Cirq (v1.5.0)	Google's open-source quantum computing framework for creating, editing, and invoking quantum circuits	Translating OpenQASM 3.0 syntax to Cirq circuits and simulating quantum algorithms	<code>cirq.Circuit()</code> , <code>cirq.Simulator()</code> , <code>cirq.ops</code> No API keys required
Qiskit (v2.1.2)	IBM's comprehensive quantum computing SDK with circuit design and simulation capabilities	Translating OpenQASM 3.0 to Qiskit circuits and executing simulations on local backends	<code>QuantumCircuit()</code> , <code>qiskit.transpile()</code> , <code>qiskit.execute()</code> No API keys required
PennyLane (v0.42.3)	Cross-platform quantum machine learning framework with automatic differentiation support	Translating OpenQASM 3.0 to PennyLane circuits for quantum ML workflows and hybrid computations	<code>qml.QNode()</code> , <code>qml.device()</code> , <code>qml.execute()</code> No API keys required
Qiskit Aer (v0.17.2)	High-performance quantum circuit simulator with multiple backend options (statevector, unitary, etc.)	Backend simulation engine providing fast and accurate quantum circuit execution for Qiskit	<code>AerSimulator()</code> , <code>Statevector()</code> , <code>UnitarySimulator()</code> No API keys required
Supporting Libraries and Frameworks			

Continued on next page

API and Version	Description	Purpose of Usage	API Endpoint/Function/Class Used
Streamlit (v1.50.0)	Modern web application framework for building interactive data science and ML applications	Creating the interactive web interface and visualization dashboard for QSim circuit editor and simulator	<code>st.sidebar()</code> , <code>st.tabs()</code> , <code>st.plotly_chart()</code> , <code>st.code_editor()</code> No API keys required
PyQASM (v0.5.0)	Python parser for OpenQASM 3.0 with AST generation and semantic analysis	Parsing and validating OpenQASM 3.0 source code into abstract syntax trees for circuit construction	<code>pyqasm.loads()</code> , <code>pyqasm.parse()</code> No API keys required
Matplotlib (v3.10.6)	Comprehensive visualization library for creating static, animated, and interactive plots	Generating quantum circuit diagrams, probability distributions, and measurement result visualizations	<code>matplotlib.pyplot</code> , <code>plt.bar()</code> , <code>plt.figure()</code> No API keys required
NumPy (v2.2.6)	Fundamental package for scientific computing with support for large multi-dimensional arrays	Performing numerical computations for quantum state vectors and matrix operations	<code>numpy.array()</code> , <code>numpy.linalg</code> , <code>numpy.random</code> No API keys required
Pandas (v2.3.2)	Data analysis and manipulation library with DataFrame structures	Managing simulation results, storing measurement data, and exporting statistical analysis	<code>pandas.DataFrame()</code> , <code>pd.read_csv()</code> , <code>df.to_csv()</code> No API keys required

Continued on next page

API and Version	Description	Purpose of Usage	API Endpoint/Function/Class Used
-----------------	-------------	------------------	----------------------------------

Table 4.2: External APIs and SDKs Used in QSim Framework

4.4 Testing Details

The QSim project maintains a comprehensive unit test suite to ensure the reliability and correctness of the OpenQASM 3.0 parser and the three supported visitor implementations: Cirq, Qiskit, and PennyLane. The tests cover a wide range of scenarios, including variable declarations, quantum gate operations, control flow, and visitor-specific translations. Besides this, complete testing has been done after integration with QCanvas, to ensure the pipelines works as intended.

Visitor SDK	Total Test Cases
Google Cirq	450+
IBM Qiskit	450+
Xanadu PennyLane	450+
Total	1400+

Table 4.3: Summary of Unit Test Coverage

Documentation Scope

Due to the extensive nature of the test suite, which comprises over 1400+ individual test cases, it is impractical to list the full source code for every test in this document. Doing so would result in hundreds of pages of code. Instead, this document provides a curated selection of representative test cases from each visitor to demonstrate the testing methodology, logic, and coverage patterns used throughout the project.

4.5 Unit Testing

4.5.1 Cirq Visitor Tests

The Cirq test suite (`cirq_test.py`) focuses on validating the `CirqVisitor`. It includes tests for circuit generation, simulation results, and specific OpenQASM 3.0 features like resets and legacy measurement syntax.

Test Case: test_R1_single_reset

Description: Verifies that a single qubit reset operation correctly sets the qubit state to $|0\rangle$. The test applies an X gate followed by a reset, then measures the qubit to confirm the result is 0.

```

1 def test_R1_single_reset():
2     """Test single qubit reset after X gate"""
3     qasm_code = """
4     OPENQASM 3.0;
5     include "stdgates.inc";
6     qubit[1] q;
7     bit[1] c;
8
9     x q[0];
10    reset q[0];
11    c[0] = measure q[0];
12    """
13    circuit, result = run_qasm(qasm_code, repetitions=10)
14    # After reset, all measurements should be 0
15    assert all(result.data["c_0"] == 0), \
16        "Reset operation failed to initialize qubit to |0>"

```

Listing 4.2: Single Qubit Reset Test Logic

Test Case: test_M1_old_single

Description: Validates the parsing and execution of the legacy OpenQASM measurement syntax (measure q -> c) for a single qubit. It ensures backward compatibility with older QASM styles.

```

1 def test_M1_old_single():
2     """Test old syntax: single qubit measurement"""
3     qasm_code = """
4     OPENQASM 3.0;
5     include "stdgates.inc";
6     qubit[1] q;
7     bit[1] c;
8
9     h q[0];
10    measure q[0] -> c[0];
11    """
12    circuit, result = run_qasm(qasm_code, repetitions=50)
13    ones = sum(result.data["c_0"])
14    # Check for roughly 50/50 distribution

```

```

15     assert 0.3 < (ones / 50) < 0.7, \
16         "Measurement distribution outside expected 50/50 range"

```

Listing 4.3: Legacy Measurement Syntax Test

Test Case: test_const_int_uint_declaration

Description: Checks the correct declaration and initialization of constant integer and unsigned integer variables. It ensures that these variables hold the correct values and are marked as constants in the symbol table.

```

1  def test_const_int_uint_declaration():
2      """Test constant int and uint declarations"""
3      qasm_code = """
4      OPENQASM 3.0;
5      const int[8] A = 42;
6      const uint[16] B = 100;
7      """
8      vars_dict = run_qasm(qasm_code)
9
10     assert vars_dict["A"]["type"] == "int", \
11         "Variable A has incorrect type"
12     assert vars_dict["A"]["value"] == 42, \
13         "Variable A has incorrect value"
14     assert vars_dict["A"]["const"] is True, \
15         "Variable A should be marked as constant"

```

Listing 4.4: Constant Declaration Test

4.5.2 Qiskit Visitor Tests

The Qiskit test suite (`qiskit_test.py`) validates the `QiskitVisitor`, ensuring that OpenQASM 3.0 code is correctly translated into Qiskit `QuantumCircuit` objects.

Test Case: test_simple_qubit_declaration

Description: Ensures that a basic qubit declaration is correctly parsed and results in a Qiskit `QuantumCircuit` with the expected number of qubits.

```

1  def test_simple_qubit_declaration(self):
2      """Test basic qubit declaration parsing"""
3      code = """
4      OPENQASM 3.0;
5      qubit[2] q;

```

```

6      """
7      visitor = QiskitVisitor()
8      visitor.visit(parse_openqasm3(code))
9      qc = visitor.finalize()
10
11     assert isinstance(qc, QuantumCircuit), \
12         "Visitor did not produce a QuantumCircuit"
13     qubit_var = visitor.get_var('qubits', 'q')
14     assert qubit_var is not None, "Qubit variable not found"
15     assert qubit_var.size == 2, "Qubit array has wrong size"
16     assert qc.num_qubits == 2, "Circuit has wrong number of
        qubits"

```

Listing 4.5: Qubit Declaration Test

Test Case: test_arithmetic_expressions

Description: Tests the evaluation of various arithmetic expressions (addition, subtraction, multiplication, division, modulo, power) within the QASM code. It verifies that the visitor correctly computes these values during traversal.

```

1  @pytest.mark.parametrize("expression, expected", [
2      ("10 + 5", 15),
3      ("10 - 3", 7),
4      ("10 % 3", 1),
5      ("2 ** 4", 16),
6      ("20 / 4", 5.0)
7  ])
8  def test_arithmetic_expressions(self, expression, expected):
9      """Test arithmetic expression evaluation in visitor"""
10     code = f"""
11     OPENQASM 3.0;
12     float[64] result = {expression};
13     """
14     visitor = QiskitVisitor()
15     visitor.visit(parse_openqasm3(code))
16     result_val = visitor.get_var('env', 'result')
17     assert result_val is not None, "Result variable not found"
18     assert math.isclose(result_val, expected), \
19         f"Expected {expected}, got {result_val}"

```

Listing 4.6: Arithmetic Expression Test

Test Case: test_simple_measurement

Description: Verifies that a standard measurement operation is correctly translated into a Qiskit measurement instruction, mapping specific quantum bits to classical bits.

```

1 def test_simple_measurement(self):
2     """Test measurement operation translation"""
3     code = """
4     OPENQASM 3.0;
5     qubit[2] q;
6     bit[2] c;
7     c[0] = measure q[0];
8     """
9     visitor = QiskitVisitor()
10    visitor.visit(parse_openqasm3(code))
11    qc = visitor.finalize()
12
13    # Check for measure operation in circuit data
14    measure_ops = [instr for instr in qc.data
15                   if instr.operation.name == "measure"]
16    assert len(measure_ops) >= 1, \
17        "No measurement operations found in circuit"
18    assert len(measure_ops) == 1, \
19        f"Expected 1 measurement, found {len(measure_ops)}"

```

Listing 4.7: Simple Measurement Test

4.5.3 PennyLane Visitor Tests

The PennyLane test suite (`pennylane_test.py`) validates the `PennyLaneVisitor`, focusing on the creation of valid PennyLane QuantumTape objects and the correct handling of operations.

Test Case: test_pennylane_visitor_simple_circuit

Description: Confirms that a simple QASM program with qubit declarations is correctly visited and results in a valid PennyLane QuantumTape.

```

1 def test_pennylane_visitor_simple_circuit(self):
2     """Test PennyLane visitor with simple circuit"""
3     code = """
4     OPENQASM 3.0;
5     qubit[2] q;
6     """
7     visitor = PennyLaneVisitor()

```

```

8     visitor.visit(parse_openqasm3(code))
9     circuit = visitor.finalize()
10
11     assert isinstance(circuit, qml.tape.QuantumTape), \
12         "Visitor did not produce a QuantumTape"
13     qubit_var = visitor.get_var('qubits', 'q')
14     assert qubit_var == [0, 1], \
15         f"Expected qubits [0, 1], got {qubit_var}"

```

Listing 4.8: Simple Circuit Test

Test Case: `test_classical_declarations_initialized`

Description: Checks that classical variables (int, float, bool) are correctly initialized in the environment when parsing QASM code. This is crucial for hybrid quantum-classical logic.

```

1 def test_classical_declarations_initialized(self):
2     """Test classical variable initialization in visitor"""
3     code = """
4     OPENQASM 3.0;
5     int[8] a = -10;
6     float[64] c = 3.14;
7     bool b = true;
8     """
9     visitor = PennyLaneVisitor()
10    visitor.visit(parse_openqasm3(code))
11
12    var_a = visitor.get_var('env', 'a')
13    var_c = visitor.get_var('env', 'c')
14    var_b = visitor.get_var('env', 'b')
15
16    assert var_a == -10, f"Expected a = -10, got {var_a}"
17    assert math.isclose(var_c, 3.14), \
18        f"Expected c = 3.14, got {var_c}"
19    assert var_b is True, f"Expected b = True, got {var_b}"

```

Listing 4.9: Classical Declaration Test

Test Case: `test_openqasm3_measurement`

Description: Validates that measurement operations are correctly identified and added to the PennyLane tape. PennyLane handles measurements differently (often as return types or mid-circuit measurements), and this test ensures `MidMeasureMP` is used.

```
1 def test_openqasm3_measurement(self):
2     """Test measurement operation in PennyLane visitor"""
3     code = """
4     OPENQASM 3.0;
5     qubit[2] q;
6     bit[2] c;
7     c[0] = measure q[0];
8     """
9     visitor = PennyLaneVisitor()
10    visitor.visit(parse_openqasm3(code))
11    tape = visitor.finalize()
12
13    measure_ops = [op for op in tape.operations
14                   if isinstance(op,
15                                qml.measurements.MidMeasureMP)]
15    assert len(measure_ops) >= 1, \
16           "No measurement operations found in tape"
17    assert len(measure_ops) == 1, \
18           f"Expected 1 measurement, found {len(measure_ops)}"
```

Listing 4.10: PennyLane Measurement Test

CHAPTER 5

CONCLUSIONS AND FUTURE WORK

5.1 Conclusion

This report presents the design, implementation, and testing of QSim, a unified quantum circuit simulation platform built to address the fragmentation and interoperability challenges prevalent in the current quantum computing landscape. By adopting OpenQASM 3.0 as a universal intermediate representation, the platform successfully decouples circuit authorship from execution, enabling users to write quantum code once and simulate it across Qiskit, Cirq, and PennyLane backends without framework-specific rewrites.

The core IR pipeline, encompassing OpenQASM 3.0 parsing, AST traversal via the visitor pattern, backend-specific circuit construction, and result serialization, was implemented and validated through a test suite of over 1400 unit tests spanning all three supported visitors. The microservice architecture, built on Kubernetes, RabbitMQ, PostgreSQL, and Amazon S3, provides a scalable and fault-tolerant execution environment with full job lifecycle management, from submission and queuing through execution, result persistence, and observability.

The FastQSim SDK further lowers the barrier to entry by exposing a clean, backend-agnostic Python interface that abstracts all REST communication, supports both synchronous and asynchronous job submission, and provides transparent access to large simulation artifacts via presigned URLs. Together, these components deliver on the platform's core objective: democratizing access to multi-framework quantum simulation without dependency on proprietary cloud infrastructure.

5.2 Future Work

FastQubit establishes a functional end-to-end cloud-native quantum simulation platform, but several directions remain open for extension and improvement.

5.2.1 Semantic-Aware Resource Scheduling (FAST-QSim)

The current scheduler treats every submitted quantum job as a black box, assigning fixed Kubernetes pod sizes regardless of circuit structure. This leads to chronic over-provisioning for small circuits and Out-Of-Memory (OOM) failures for large ones. A planned research contribution, *FAST-QSim*, addresses this by introducing a **static liveness analyzer for OpenQASM 3.0** that computes the peak live qubit count N_q^{peak} through four analysis passes—register collection, alias resolution, call-graph unification, and liveness scanning—correctly handling subroutine argument passing, `let` qubit aliases, `reset`-based reuse, and recursive gate definitions that cause naive register-counting to produce incorrect estimates. Given N_q^{peak} , the statevector memory requirement $M_{\text{est}} = \alpha \cdot 16 \cdot 2^{N_q^{\text{peak}}} + \beta$ is computed deterministically and the job is routed to the smallest of six pre-warmed **memory buckets** (64 MB through 2 GB), eliminating both OOM failures and the ≈ 1 s Kubernetes cold-start penalty for in-scope jobs through process-level pod recycling (≈ 100 ms). This work is planned as a formal research publication targeting circuits with $N_q \leq 27$ in double-precision statevector mode, where memory guarantees are exact and unconditional.

5.2.2 Circuit Profiling and Resource Estimation

Two fields in the job data model—`profile_json` and `estimate_json`—are currently always null. Future work includes wiring the OpenQASM 3.0 AST analyzer to populate these fields with circuit metadata (gate counts, depth, two-qubit gate density, T-count) and using them to generate realistic CPU and wall-time estimates before a job enters the queue. These estimates would unlock cost-aware job prioritisation, per-circuit billing breakdowns, and user-facing circuit complexity feedback in the QCanvas interface.

5.2.3 Job Cancellation and Preemption

Cancel requests currently mark the database record only; a running Q-Worker pod may complete and attempt to write results that are then silently discarded. A proper cancellation mechanism requires a kill signal delivered to the running pod—via a Kubernetes `Job` deletion or an out-of-band Redis pub/sub channel consumed by the worker—so that cluster resources are freed immediately on user request. This is particularly important for long-running large-qubit simulations occupying high-memory pods.

5.2.4 Quota and Billing Reconciliation

Redis concurrency counters can drift from actual pod counts following unexpected pod crashes, leaving the quota system in an inconsistent state until the next maintenance sweep. Future work should introduce a continuous reconciliation loop with shorter intervals, and explore replacing the current approximation with an event-driven approach where each pod lifecycle event (start, completion, OOM kill) emits an authoritative counter update.

5.2.5 Multi-Backend and GPU Simulation

FastQSim Core currently supports Qiskit Aer, Cirq, and PennyLane on CPU. A natural extension is **GPU-accelerated statevector simulation** via Qiskit Aer GPU or NVIDIA cuStateVec, which would reduce wall-clock time for 20–27-qubit circuits by one to two orders of magnitude on the existing GPU node pool. On the algorithmic side, adding **tensor-network (MPS)** and **density-matrix** backends for error-noise simulation would significantly expand the class of circuits FastQubit can serve beyond ideal statevector workloads.

5.2.6 Stateless Execution Latency

The session router restarts the Jupyter kernel after each `execute_reply`+idle cycle to enforce stateless execution. This adds measurable latency between sequential cell executions in QCanvas. Future work should evaluate alternatives—snapshot/restore of kernel state, namespace-level isolation without full restart, or an opt-in stateful mode for trusted users—to reduce this overhead while preserving the security isolation that motivates the current design.

5.2.7 Research Paper: The FastQubit Platform

Beyond FAST-QSim, the full FastQubit system—comprising QCanvas (visual circuit designer), the FastQSim SDK (programmatic job and session API), and the FastQubit orchestration backend—warrants a dedicated system-level research paper. Such a paper would characterise the end-to-end user workflow: from drag-and-drop circuit construction in QCanvas through WebSocket-proxied Jupyter kernel execution to asynchronous batch-job submission and S3-backed result retrieval. It would evaluate the platform on dimensions including interactive session latency, multi-tenant quota isolation under concurrent load, billing accuracy across the full job lifecycle, and SDK usability for quantum algorithm developers—situating FastQubit among cloud quantum services such as IBM Quantum, AWS Braket, and Azure Quantum while highlighting its distinguishing properties: full open-source deployability on commodity AWS infrastructure and OpenQASM 3.0-native semantic analysis.

APPENDIX A

CODE EXAMPLES

This appendix shows how the same 2-qubit Bell state circuit can be implemented in different quantum programming frameworks.

A.1 Qiskit Implementation

```
1 from qiskit import QuantumCircuit, Aer, execute
2
3 qc = QuantumCircuit(2, 2)
4 qc.h(0)
5 qc.cx(0, 1)
6 qc.measure([0, 1], [0, 1])
7
8 backend = Aer.get_backend("qasm_simulator")
9 job = execute(qc, backend=backend, shots=2048)
10 print(job.result().get_counts())
```

Listing A.1: Bell state circuit in Qiskit

A.2 Cirq Implementation

```
1 import cirq
2
3 q0, q1 = cirq.LineQubit.range(2)
4 circuit = cirq.Circuit(
5     cirq.H(q0),
6     cirq.CNOT(q0, q1),
7     cirq.measure(q0, q1, key='result')
8 )
```

```
9
10 simulator = cirq.Simulator()
11 result = simulator.run(circuit, repetitions=2048)
12 print(result.histogram(key='result'))
```

Listing A.2: Bell state circuit in Cirq

A.3 PennyLane Implementation

```
1 import pennylane as qml
2 from pennylane import numpy as np
3
4 dev = qml.device("default.qubit", wires=2, shots=2048)
5
6 @qml.qnode(dev)
7 def bell_circuit():
8     qml.Hadamard(wires=0)
9     qml.CNOT(wires=[0, 1])
10    return qml.sample(wires=[0, 1])
11
12 samples = bell_circuit()
13 print(np.unique(samples, axis=0, return_counts=True))
```

Listing A.3: Bell state circuit in PennyLane

A.4 OpenQASM 3.0 Code

```
1 OPENQASM 3.0;
2 include "stdgates.inc";
3
4 qubit q[2];
5 bit c[2];
6
7 h q[0];
8 cx q[0], q[1];
9
10 c[0] = measure q[0];
11 c[1] = measure q[1];
```

Listing A.4: Bell state circuit in OpenQASM 3.0

A.5 QSim API Example

Note: This is a prototype example; the API may change in future.

```
1
2 from qiskit import QuantumCircuit
3 from qsim import run, get_job_status, get_results
4
5 qc = QuantumCircuit(2, 2)
6 qc.h(0)
7 qc.cx(0, 1)
8 qc.measure_all()
9
10 jobID = run(circuit=qc, backend="cirq",
11             sim_type="statevector", shots=2048)
12
13 while True:
14     status = get_job_status(jobID)
15     if status == "completed":
16         results = get_results(jobID)
17         print("Statevector:")
18         print(results.statevector)
19         break
```

Listing A.5: Bell state circuit using QSim API

Bibliography

- [1] Google Quantum AI. Cirq: A python framework for creating, editing, and invoking noisy intermediate scale quantum circuits, 2023. Accessed: 2025-09-12.
- [2] Andrew W. Cross, Ali Javadi-Abhari, Thomas Alexander, and et al. Openqasm 3.0 language specification. <https://openqasm.com/versions/3.0/>, 2022. Accessed: 2025-09-12.
- [3] Xanadu Quantum Technologies Inc. PennyLane: Automatic differentiation of hybrid quantum-classical computations, 2023. Accessed: 2025-09-12.
- [4] Qiskit Development Team. Qiskit: An open-source framework for quantum computing, 2023. Accessed: 2025-09-12.